



NetSeaDF Official Handbook

Author: Nick Fran

Date Of Creation: Apr 15, 2026

Date Of Last Update: May 26, 2026

Subject: Documentation, Guide



Overview

The following is a comprehensive overview of the entire NetSeaDF application, from available functionalities, to basic usage, to code snippets of internal systems. This handbook is the ultimate guide to NetSeaDF.

The document is broken up into two fundamental components, **User information** and **Developer information**. This is to ensure that anyone can benefit and learn from this handbook, regardless of whether they are a typical end-user hoping to learn how to use the app, or a developer looking to understand the applications architecture.

Given the fact that this handbook begins with the User information section, followed by the Developer information section, End-users looking to use this application for data visualization and oceanography will benefit most from the first half. Developers will benefit most from the second half.

Although the User Section will encompass every possible action a user can perform in the app as well as all other important information, the developer section will not be as exhaustive and fully fledged.

As NetSeaDF's development came to a close, the codebase had already reached over 10,000 Lines of code (this is not including CSS or any libraries). Because of this, the developer section of this document had to be truncated in order to avoid the possibility of writing a 500 page document about every line of code. Because of this reason, the developer section has been written to adhere more to an "abstract overview" or "documentation" style format rather than an "every line of code" type of format.

Regardless, the developer section will provide a full understanding of NetSeaDF's software development life cycle as well as how data is handled and passed around in the application.



Overview.....	1
Common terminology in this guide.....	7
Purpose & Utility.....	8
User Section.....	9
Common Uses.....	10
How To Install.....	12
• Step 1: Visit the NetSeaDF GitHub page.....	12
• Step 2: Click on “Releases”.....	12
• Step 3: Select the desired software version.....	12
• Step 4: Scroll down to the “Assets” section and click on the executable.....	12
Understanding The Interface.....	13
Each button and its function.....	14
Understanding Map and View Mode.....	16
Explanation of a “View”.....	17
Application Features.....	18
Changing user settings.....	18
Importing .nc files.....	18
Selecting one/many .nc files.....	19
Viewing data in an .nc file.....	20
Finding a selected .nc file on the map.....	20
Filtering platforms by time (slider).....	21
Filtering platforms by time (manual).....	22
Filtering platforms by location.....	23
Creating Views.....	24
Customizing options when creating a view.....	25
Filtering plot variables.....	26
Plotting.....	27
Saving Views.....	27
Data Transfer.....	27
Viewing app notifications and errors.....	28
Saving views.....	28
Transferring data.....	28
Online and Offline Maps.....	29
How to create a view.....	30
How plotting works in NetSeaDF.....	31
Chart Settings.....	32
General Settings.....	32
The Axis Settings.....	33
View Generation.....	34
View Output.....	34

View Saving.....	35
The “Include Plot Data” Option.....	35
In-App Notifications.....	36
Logs.....	37
Log Classifications.....	38
How to transfer user data between NetSeaDF installations.....	40
Exporting transferred data.....	40
Importing transferred data.....	42
User Settings.....	43
Troubleshooting.....	44
Help: Rebooting.....	44
Help: Notifications.....	44
Help: Logging.....	44
Help: Tooling.....	45
Common Issues.....	46
File Desync.....	46
Developer Section.....	47
Software used in development.....	48
Tech Stack.....	49
Tech Stack Explanation.....	49
SDL (Software Development Lifecycle).....	51
Stage: Expectations.....	51
Stage: Planning.....	51
Stage: Design.....	51
Stage: Architecting.....	52
Stage: Implementation.....	52
Stage: Pre-Production.....	52
Stage: Production.....	52
map.html (The most important file).....	53
CI/CD Workflow.....	56
Modules.....	57
Config.js.....	57
pathDep.js.....	58
fileHandle.js.....	60
queue.js.....	62
transfer.js.....	63
integrations.js.....	64
pythonProcess.py.....	66
DOM.js.....	68
userInteractions.js.....	70
charts.js.....	71

logging.js.....	72
objects.js.....	74
popup.js.....	75
preload.js.....	76
basicFunctions.js.....	77
Handling of application state.....	78
Python + JavaScript Integration.....	80
Anatomy of calling a PY function from a JS frontend.....	82
simpleData.json (Dataset Data Representation).....	87
Use of AI in NetSeaDF.....	92
AI: Models Used.....	92
AI: Context.....	93
Special Thanks.....	99

Common terminology in this guide

The following includes specific terminology in the application that refers to concepts within the field of oceanography, computers and acoustics. It is of utmost importance that individuals interacting with this document understand the following terms so that a base line of understanding and communication can be established.

Term	Meaning
ARGO	A major international global observation program that uses a fleet of nearly 4,000 autonomous, free-drifting robotic floats to measure temperature and salinity in the ocean.
NetSeaDF	The name of the application.
NetCDF	A standardized file format used extensively in oceanography for storing scientific data.
Timestamp	Sometimes used interchangeably with an individual "Profile". It represents the specific date and time for when an observation was recorded.
SSP	A measurement or model of how the speed of sound changes with depth in the water column. This is critical for underwater navigation and acoustic communication. Sound speed is heavily influenced by temperature, salinity, and pressure.
Coords	Numerical values that define the location of data points in space and time (e.g., latitude, longitude, depth/pressure, and time). (NetSeaDF typically just thinks of coords in terms of latitude and longitude)
Vars and Dims	In data formats like NetCDF, Dimensions define the "shape" of the data (e.g., time, depth, lat, lon), while Variables are the actual data arrays (e.g., temperature, salinity) that are mapped across those dimensions.
Plot	The visual representation of oceanographic data. In this context, it often refers to vertical profile plots of oceanographic variables.
Platform	A general term for any structure or vehicle used to collect data. This can include ships, moored buoys, satellites, underwater gliders, or drifting floats.
Float	An autonomous, battery-powered oceanographic instrument that drifts with currents. It typically performs "cycles" by sinking to a target depth, drifting, and then rising to the surface to transmit its collected data via satellite.

Purpose & Utility

The sole purpose of this software application is to facilitate better data exploration and data discovery with respect to oceanography and acoustics.

There is a lot of oceanographic data, too much for a single person to conceptualize and handle. Applications have to be constructed to help scientists and researchers better manage this data.

The application is centered around data specific to ARGO floats, and the application expects that data to be in NetCDF format. Given the fact that there are thousands of floats on ARGO's web server, each with anywhere from 2-100+ timestamps, each of which contains potentially more than a handful of variables measured all the way down to 2000+ meters, It just simply is not feasible for a human to comprehend and make sense of every pattern in this data that might be hiding. Tools need to be constructed to visualize this data in different ways to suit current and future research and for this reason, NetSeaDF was created.

User Section

Common Uses

NetSeaDF is a complex program that allows the user to do many different things. Some examples of common uses for the application are as follows.

Visualizing Float Profile Timelines.

Looking deeper into individual floats and visualizing their profile timelines as well as timestamps to determine where it went, and perhaps why.

Map Visualizations

Visualizing imported ARGO floats on a map, and inferring from this geospatial data which areas of the ocean have many datasets that a user has imported, and which areas are lacking any substantial datasets, if any at all.

Under Ice Evaluations

Evaluating the possibility of floats that drifted under ice by looking at the duration in between each individual timestamp to see if any timestamp took significantly longer to happen than was expected.

SSP plotting throughout the water column

Visualizing a dataset's sound speed profile with respect to decibaric pressure across each of its timestamps.

SSP and TEMP contrasting plots

Contrasting a dataset's sound speed profile with its temperature, both with respect to decibarc pressure.

Simple SSP plotting across a time range

Visualizing sound speed profiles for first-timestamps across any number of datasets which occurred during a specific time range.

How To Install

- Step 1: Visit the NetSeaDF GitHub page
- Step 2: Click on “Releases”
- Step 3: Select the desired software version
- Step 4: Scroll down to the “Assets” section and click on the executable



Step 1

NetSeaDF Public

Unpin Unwatch 0 Fork 0 Star 0

main 2 Branches 5 Tags Go to file + <> Code

About

NetSeaDF, a cross-platform web-based desktop application, revolving around the use of mapping and data visualization for oceanographic datasets (especially for ARGO float acoustic datasets).

oceanography multipplatform acoustics

Readme BSD-2-Clause license Activity 0 stars 0 watching 0 forks

Releases 4

v1.3.0 (Latest) 2 weeks ago + 3 releases

Contributors 1 NickFran Nick

Step 2

Mac Bug fix (v1.0.0) 2 months ago

Settings menu functional and nearly done last month

Cleaning up log calls in map.html last month

queues folder not in prod (fix) last month

temp files added to init file system 4 months ago

Sonet remove queue feature 2 months ago

v1.0.5 last month

Styling fix for settings menu on small screens last month

frontend work, some backend 2 months ago

Initial commit 4 months ago

windows venv not built properly in prod, now ... last month

logging system addition last month

acoustics

Readme BSD-2-Clause license Activity 0 stars 0 watching 0 forks

Releases 4

v1.3.0 (Latest) 2 weeks ago + 3 releases

Contributors 1 NickFran Nick

Step 3

NickFran / NetSeaDF

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Releases Tags Draft a new release Find a release

2 weeks ago NickFran v1.3.0 89F3c3b Compare

v1.3.0 (Latest)

v1.3.0

Settings and Configuration

Platform Settings

Basics Debug

enableDebug false

DebugFS true

DebugEvents true

DebugInput true

Step 4

Features

- Settings Menu

A new settings menu has been added to the top of the interface, allowing users to customize the application to their personal preferences. Currently, the settings menu only includes a setting for the maps default zoom level, as well as enabling debug mode. Settings automatically update within the apps config files as changes are made without the need for an apply button. Settings persist over time between different app sessions and more settings will be added soon.

Up and coming features

- Plotting Features
- Note-Taking support for imported files to add file-specific notes.

Assets 4

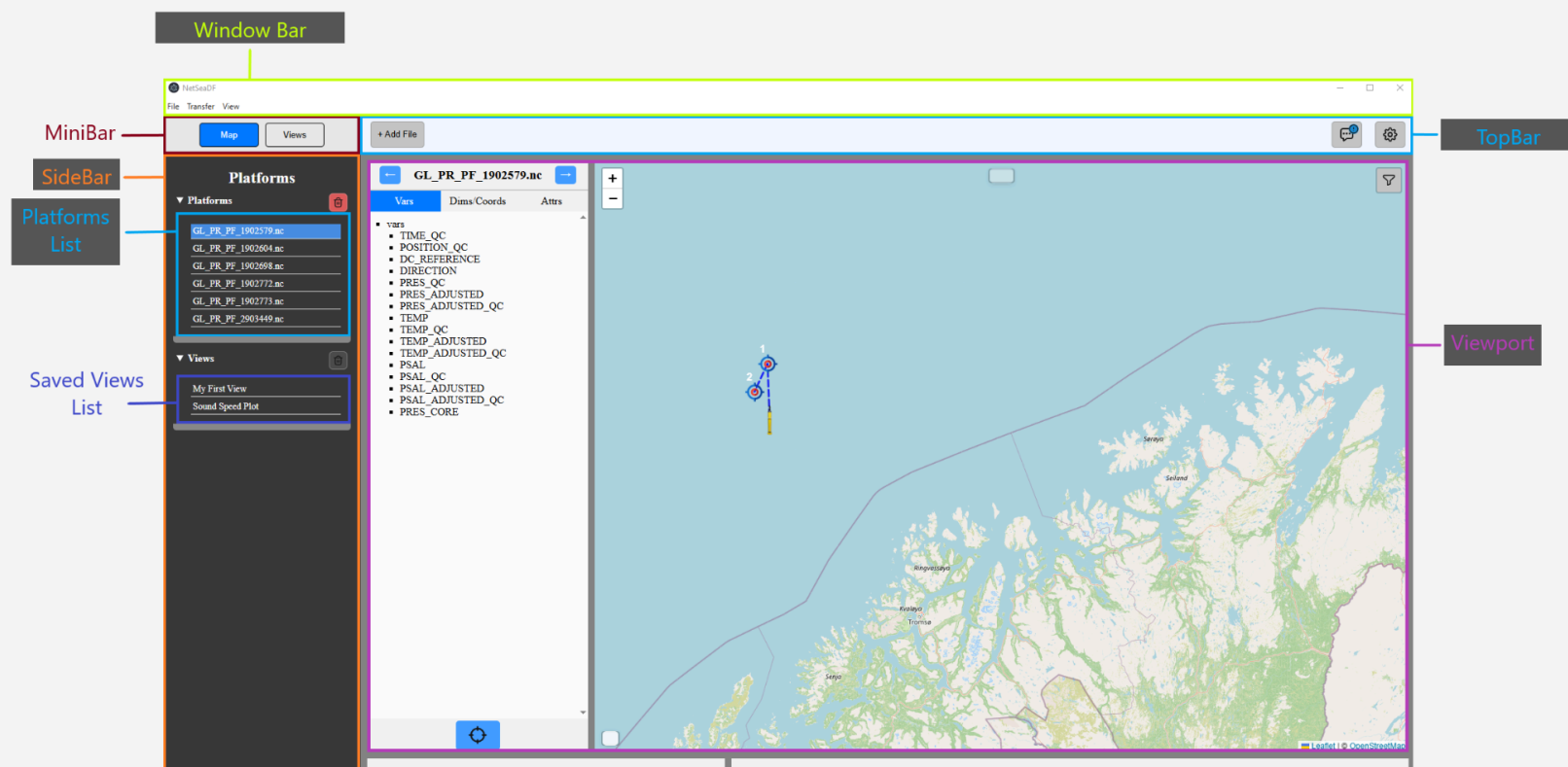
Asset	SHA256	Size	Time
NetSeaDF-v1.3.0-arm64.dmg	sha256:967c85bc55157c8ba1051662ca28...	328 MB	2 weeks ago
NetSeaDF-v1.3.0-Setup.exe	sha256:1b89de98c056169d8e0d04ac9f1ef...	179 MB	2 weeks ago
Source code (zip)			last month
Source code (tar.gz)			last month

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share your personal information

Understanding The Interface

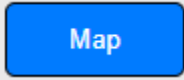
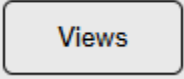
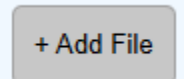
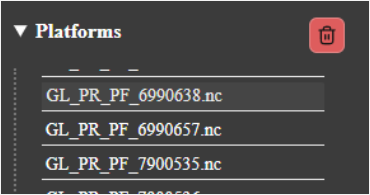
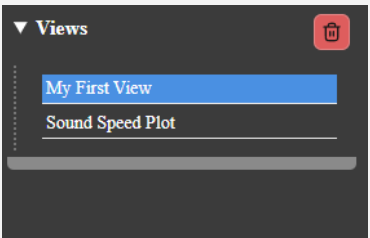
The application is broken up into several components that all serve different purposes. The following are all the main components of the interface.

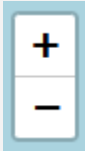
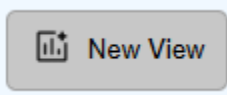
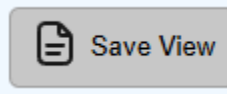
- **Window Bar**
- **MiniBar**
- **TopBar**
- **SideBar**
 - Platforms List
 - Saved Views List
- **Viewport**



Each button and its function

The following is a list of every button in NetSeaDF and what it does.

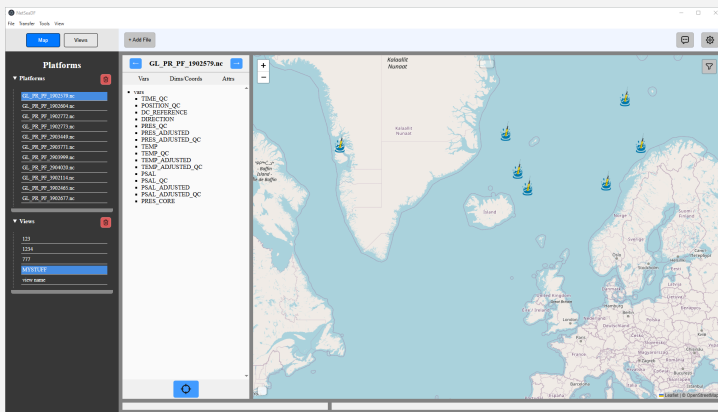
Button Name	Icon	Description
Map Mode Button		Button for changing the application to map mode.
View Mode Button		Button for changing the application to view mode.
Add File Button		Button for importing files.
Notifications Button		Button for opening the notifications menu.
Settings Button		Button for opening the settings menu.
Platforms List Delete Button		The list in the sidebar for all imported platforms.
Saved Views List Delete Button		The list in the sidebar for all saved views.
Selection Arrows		Arrows for going forwards and backwards in your group of selected files.

Selection Focus Button		Button for focusing the map on the currently selected marker.
Map Filter Button		Button to enter the map's filter mode.
Map Zoom Buttons		Buttons to zoom the map in and out.
Time Preference Dropdown	Timeline Preference Manual Slider ▾	Dropdown menu for changing between the two time filter types.
Filter Type Dropdown	Filter Type: Time ▾	Dropdown menu for switching between Time and Location based filters.
Apply Filter Button	Apply Filter	Button in the manual input time filter menu to apply the manual range.
View Profile Timeline Button	Show Profile Timeline	Button on map markers to show the profile timeline/history.
New View Button		Button to initialize a new view
Create View Button	CREATE VIEW	Button to finalize the creation of a new view.
Save View Button		Button to save the current view to a file.

Understanding Map and View Mode

All things that can be done within the application happen within two different modes of the software. These modes are as follows.

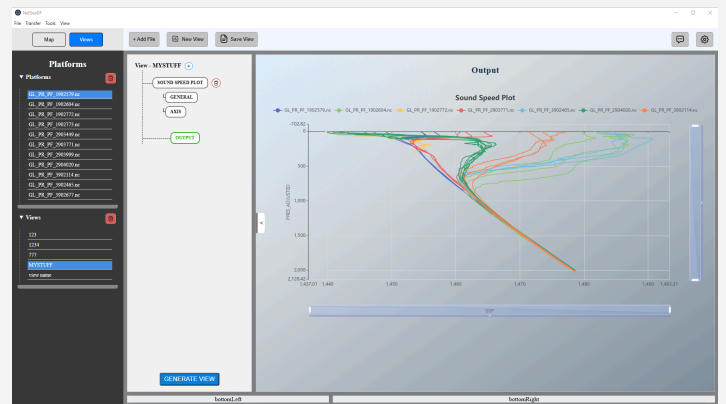
Map Mode



Holds and displays all buttons, functions and interactivity for using the applications map.

This is where you go to look at float locations.

View Mode



Holds all functionality for creating and visualizing plots and graphs.

This is where you go to work with the actual data for plotting.

Users can switch between these two modes by using the two buttons in the top left of the app.



Explanation of a “View”

A large part of NetSeaDF is plotting. In order to satisfy this need, substantial work was put into how plots will be represented in the application and how this feature will work.

The way plotting is implemented within NetSeaDF can be understood by understanding what a “View” is.

A view in NetSeaDF is a visualization within which all your plots reside. It is the thing surrounding your plots and acts as a bit of an abstract concept. But there isn't such a thing as a plot on its own in netSeaDF, all plots exist within a view because the view is what encapsulates them.

You can think of a view as a “plotting session” if you will.

Even if only one plot has been constructed, that plot still exists within a view. Therefore when you save your plots, what you are actually saving is the view of those plots.

A view contains many things including the files used in the view, as well as chart instances, and those chart instances contain the data each chart needs to function and display.

Part of the reason why this idea of a view came about was through the expectation that multiple plots would need to be displayed side-by-side. Therefore, the idea of creating a “plot container” for these plots became a necessity so that multiple plots can exist together and never be separated.

In the following pages, this feature will be brought up in more detail.

Application Features

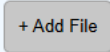
The following is a comprehensive overview of each of the primary features of NetSeaDF.

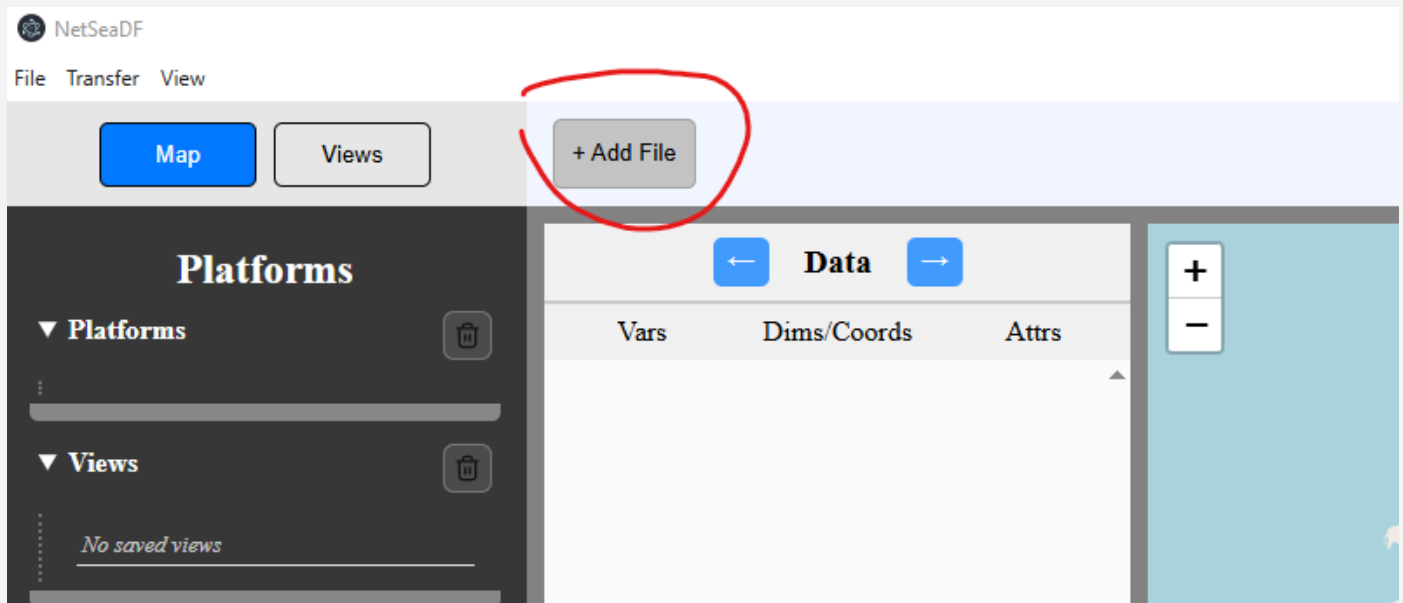
Changing user settings

By clicking the  icon in the top right of the app, users can customize their application to better suit their needs.

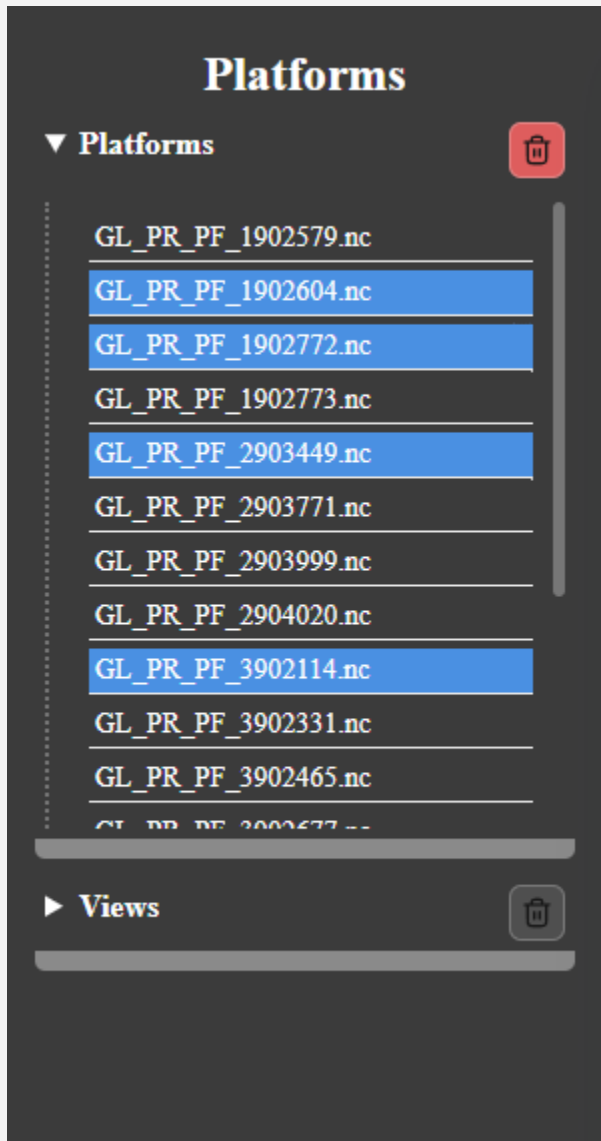
Some of these settings include customization for importing and exporting, debugging and basic settings. The list of common settings and what they control are described later on in this document.

Importing .nc files

Clicking the  Button will cause a window to appear, prompting the user to select files for import. Any number of files can be selected for an import, but they always must be .nc files.



A loading screen will appear once the user has finalized their imports. Once files are imported, they will appear in the sidebar.



Selecting one/many .nc files

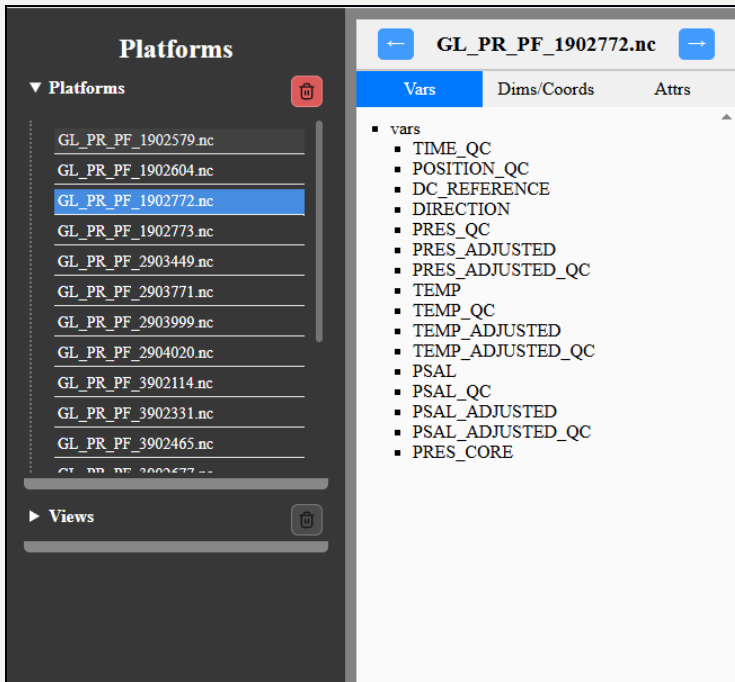
By holding the “SHIFT” key on your device and then clicking on a file in the platforms list, you can multi-select multiple files from the list.

By clicking without shift, you can return to individually selecting again.

When multi-selecting, you can use the arrow buttons to scroll through your selection, allowing you to view information on the currently focused platform within your entire selection. (this also works for the Platform Focus Button)

Viewing data in an .nc file

After selecting files in the platform list, you can look in the Info Panel on the left of the map to see a substantial amount of data about the file.



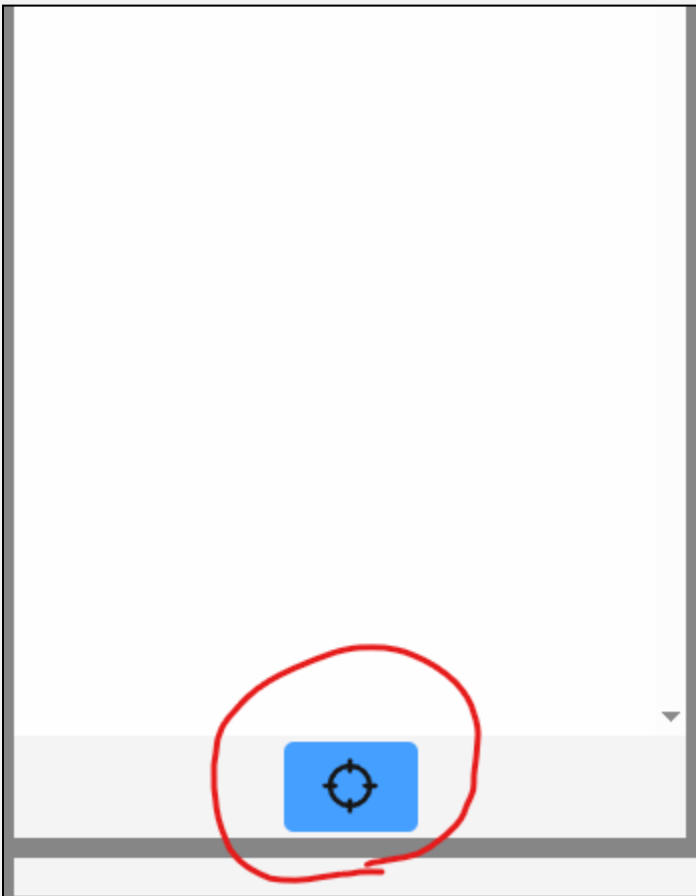
By clicking on the 3 Tabs at the top of the panel, you can switch between the variables, dimensions & coords as well as the attributes of the file.

By using the Selection Arrows, you can see the data for each selected file as you scroll through your selection.

Finding a selected .nc file on the map

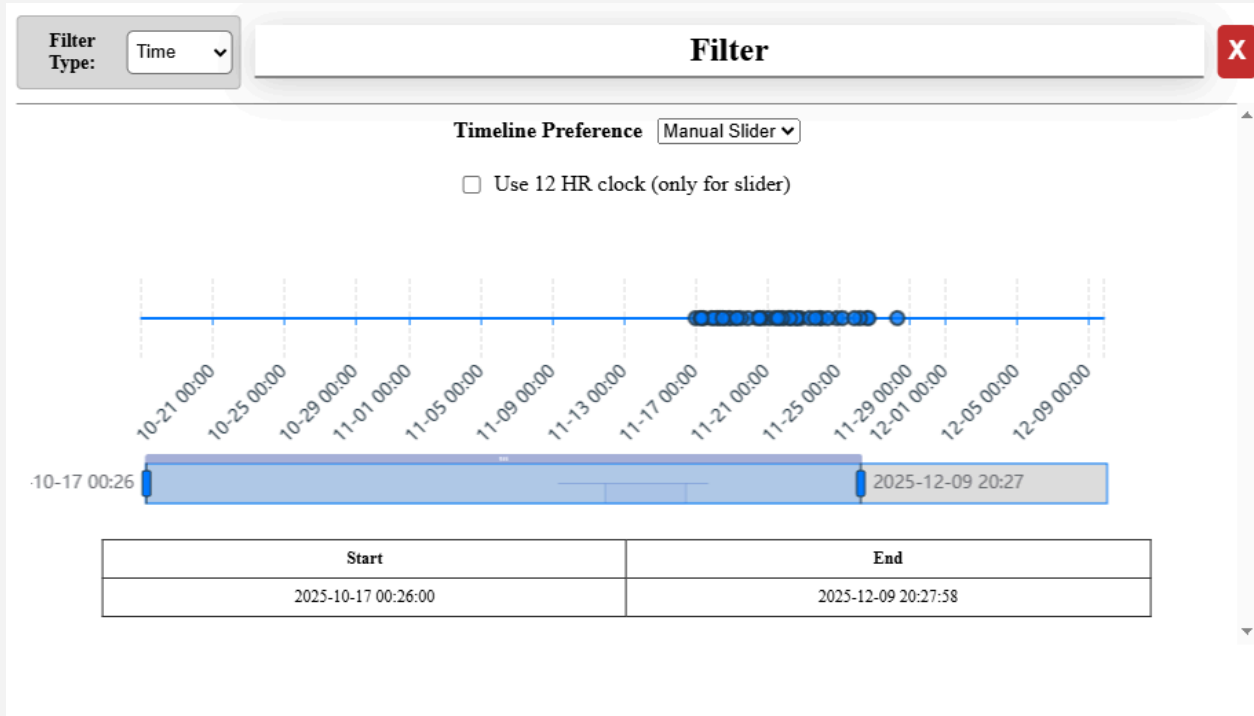
By using the Selection Focus Button at the bottom of the info panel, you can make the map auto slide to the location of the selected platform.

This makes it easy to find platforms on the map.

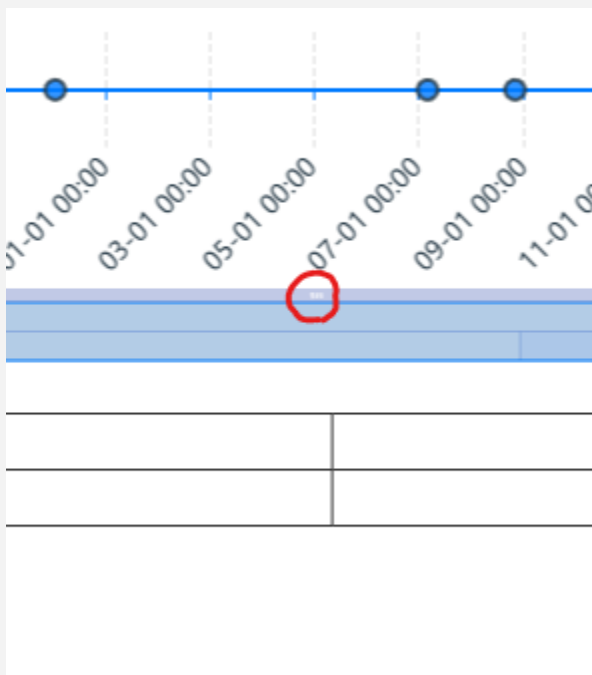


Filtering platforms by time (slider)

By opening the map filter and sliding the timeline, you can adjust the filtering of the platforms based on the start and end datetime of the slider.



The blue dots on the blue line represent the date (timestamp) of the first profile of each file that has been imported into the map. (The slider you see in the above picture was of .nc files during one week in Nov 2025).



By holding your mouse over the top edge of the zoom bar, you can slide your range across the timeline.

Filtering platforms by time (manual)

Timeline Preference Manual Input ▼

By changing the “Timeline Preference” to “Manual input”, you can switch to a more manual type of time filtering.

By clicking on the calendar icons, you can open up a sub menu, this sub menu allows you to have complete control of the filtered datetime, down to single minutes.

	Start	End
Input	mm/dd/yyyy --:-- -- 	mm/dd/yyyy --:-- -- 
<button>Apply Filter</button>		

	Start																																																																																				
Input	11/01/2025 01:00 PM 																																																																																				
November 2025 ▼ ↑ ↓ <table border="1"> <thead> <tr> <th>Su</th><th>Mo</th><th>Tu</th><th>We</th><th>Th</th><th>Fr</th><th>Sa</th></tr> </thead> <tbody> <tr> <td></td><td></td><td></td><td></td><td></td><td></td><td>1</td></tr> <tr> <td>26</td><td>27</td><td>28</td><td>29</td><td>30</td><td>31</td><td></td></tr> <tr> <td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td></tr> <tr> <td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr> <tr> <td>16</td><td>17</td><td>18</td><td>19</td><td>20</td><td>21</td><td>22</td></tr> <tr> <td>23</td><td>24</td><td>25</td><td>26</td><td>27</td><td>28</td><td>29</td></tr> <tr> <td>30</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td></tr> </tbody> </table> Clear Today <table border="1"> <tr> <td>01</td><td>00</td><td>PM</td><td>Apply Filter</td></tr> <tr> <td>02</td><td>01</td><td>AM</td><td></td></tr> <tr> <td>03</td><td>02</td><td></td><td></td></tr> <tr> <td>04</td><td>03</td><td></td><td></td></tr> <tr> <td>05</td><td>04</td><td></td><td></td></tr> <tr> <td>06</td><td>05</td><td></td><td></td></tr> <tr> <td>07</td><td>06</td><td></td><td></td></tr> </table>		Su	Mo	Tu	We	Th	Fr	Sa							1	26	27	28	29	30	31		2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	1	2	3	4	5	6	01	00	PM	Apply Filter	02	01	AM		03	02			04	03			05	04			06	05			07	06		
Su	Mo	Tu	We	Th	Fr	Sa																																																																															
						1																																																																															
26	27	28	29	30	31																																																																																
2	3	4	5	6	7	8																																																																															
9	10	11	12	13	14	15																																																																															
16	17	18	19	20	21	22																																																																															
23	24	25	26	27	28	29																																																																															
30	1	2	3	4	5	6																																																																															
01	00	PM	Apply Filter																																																																																		
02	01	AM																																																																																			
03	02																																																																																				
04	03																																																																																				
05	04																																																																																				
06	05																																																																																				
07	06																																																																																				

Once you are done setting the filter, click the apply button to apply the filter.

Note: (When using the manual input time filter, both start and end datetimes have to be set, you cannot only set one of them, it has to be both).

Note: (The timeline slider and the calendar type won't interfere with each other. You can adjust both, the one that takes effect is the one you have most recently interacted with)

Filtering platforms by location

Filter Type: Location ▼

By changing the Filter Type dropdown from “Time” to “Location”, you can filter your timelines also by location.

Using the 4 positional inputs, you can shape your filter boundary to anywhere on earth.

N	-90
S	90
W	-180
E	180

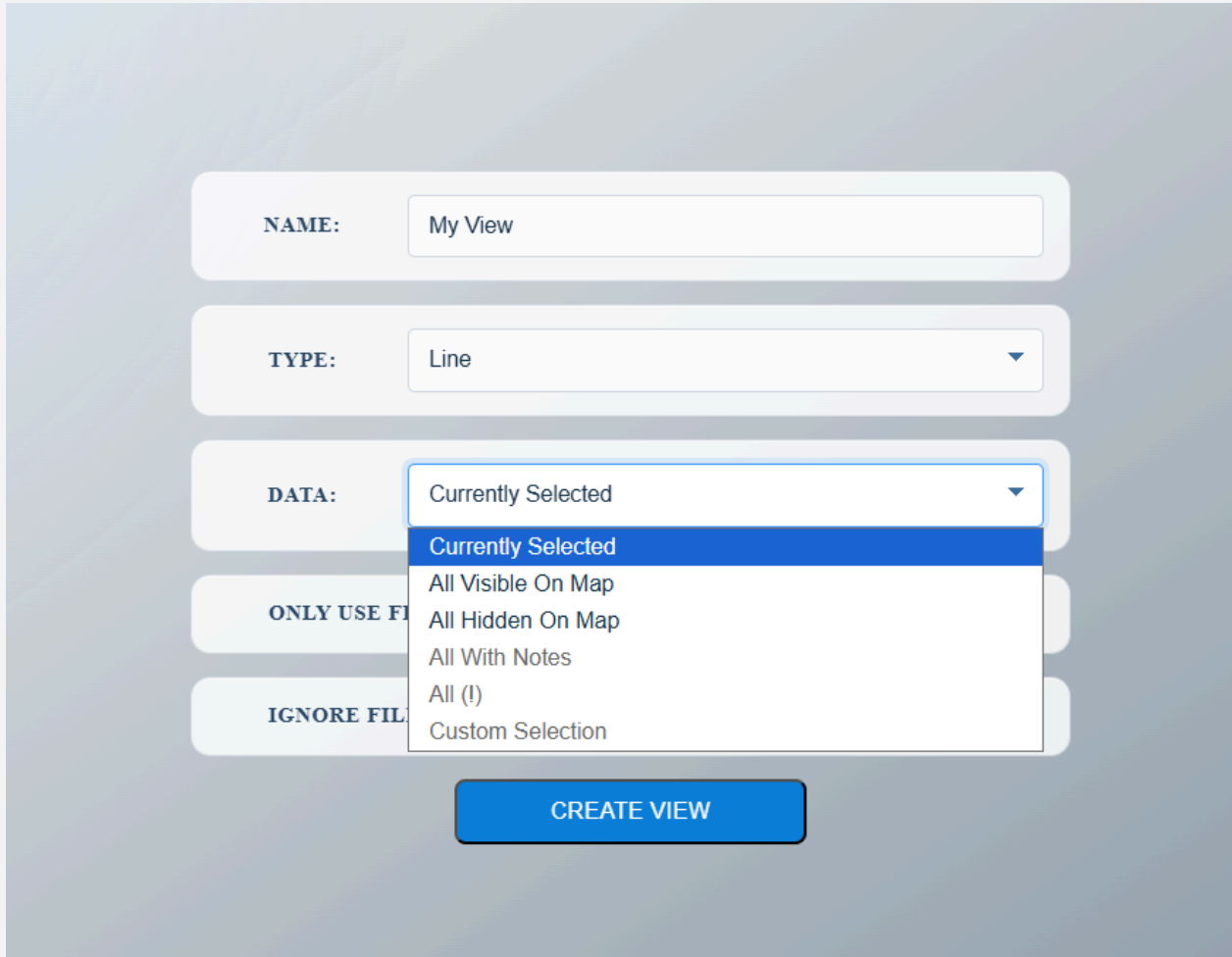
By location filtering, you get a map that displays a boundary of your selection.



(This example is using N:50,S:20,W:0,E:180)

Creating Views

By changing to view mode and clicking on the “New View” Screen, you can assemble a view.



NAME: My View

TYPE: Line

DATA: Currently Selected

ONLY USE F

IGNORE FIL

CREATE VIEW

This is the basis for creating views in NetSeaDF.

By creating a view, it allows you to set up charts and configure them to display plots.

Customizing options when creating a view

Name:

The user-defined label for the view. This helps identify and organize saved views for later retrieval and sharing.

Type:

- The chart or visualization type for the view. Common options include:
 - Line (only option at the moment)

Data:

Specifies which datasets will be included in the view. Options typically are:

- Currently Selected: Only the files currently selected by the user.
- All Visible On Map: All datasets currently visible on the map.
- All Hidden On Map: Datasets that are hidden from the map view.

Only Use First Timestamp/s:

If checked, the view will only use the first timestamp from each dataset, useful for summary or snapshot visualizations.

Ignore Files Without PRES/PSAL/TEMP:

If checked, it only includes files that contain all three key variables: Pressure (PRES), Salinity (PSAL), and Temperature (TEMP) (necessary for SSP plotting).

Edit Filtered Variables:

Opens a popup to manage a list of variables to exclude from axis selection in the view.

Filtering plot variables

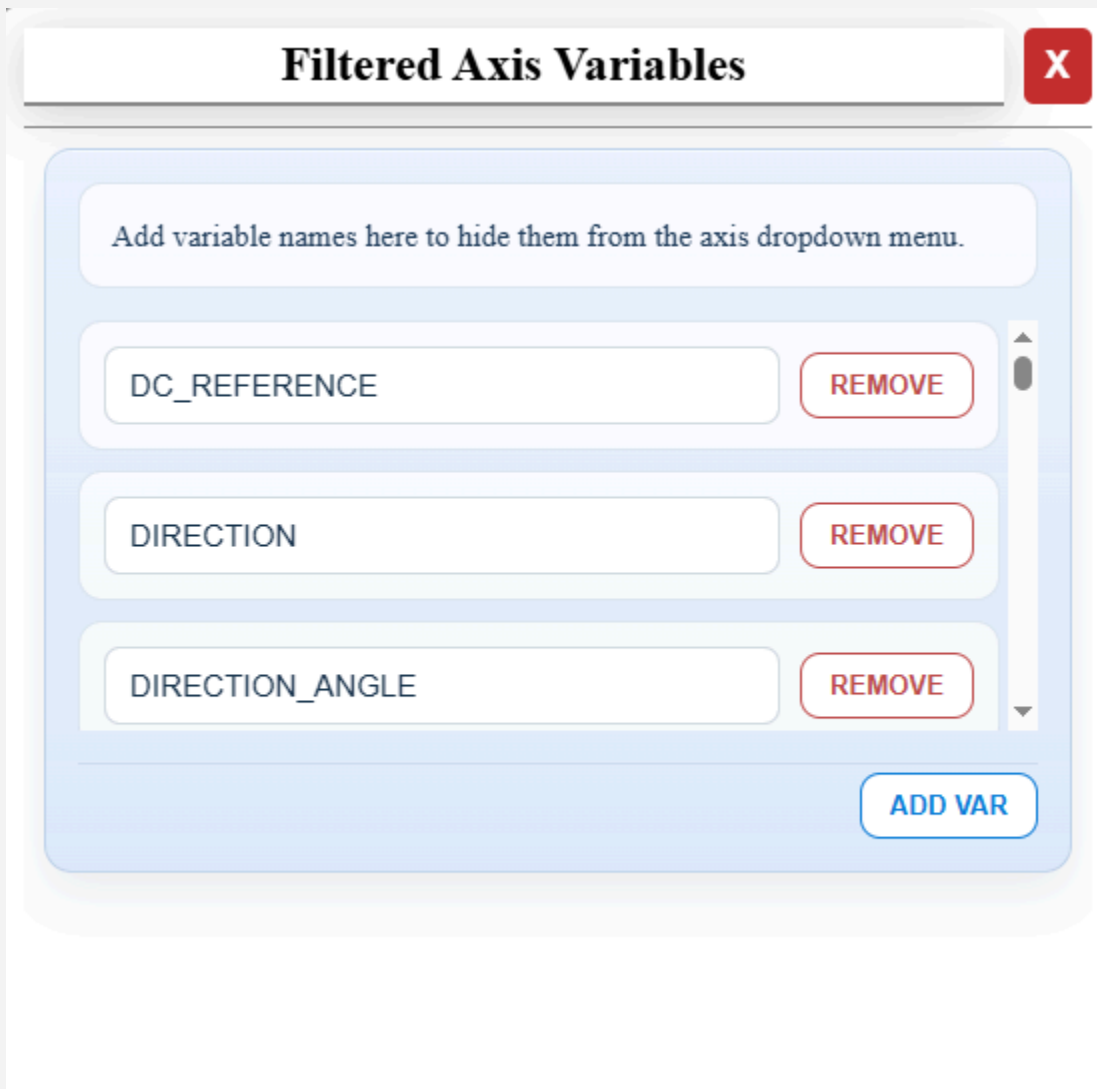
Some .nc files have over 30 variables, that's a lot to sift through to find the ones you need when you are creating plots. Selecting SSP for the x axis when creating a chart is fine, but it's a headache if there are tons of other variables you don't need to see.

And for this reason, the var blacklist filter was created.

This feature provides a small menu for users to interact with and allows users to add as many variables as they want to the black list. Any variables in the blacklist will not show up as an option for the axis of plots, ensuring when the user attempt to set a variable for a plot, they can minimize unwanted variables

The menu is simple, and lacks the need for an explanation.

(The only thing to be aware of is that this filtered list is available to transfer using the data transfer feature).



Filtered Axis Variables X

Add variable names here to hide them from the axis dropdown menu.

DC_REFERENCE

REMOVE

DIRECTION

REMOVE

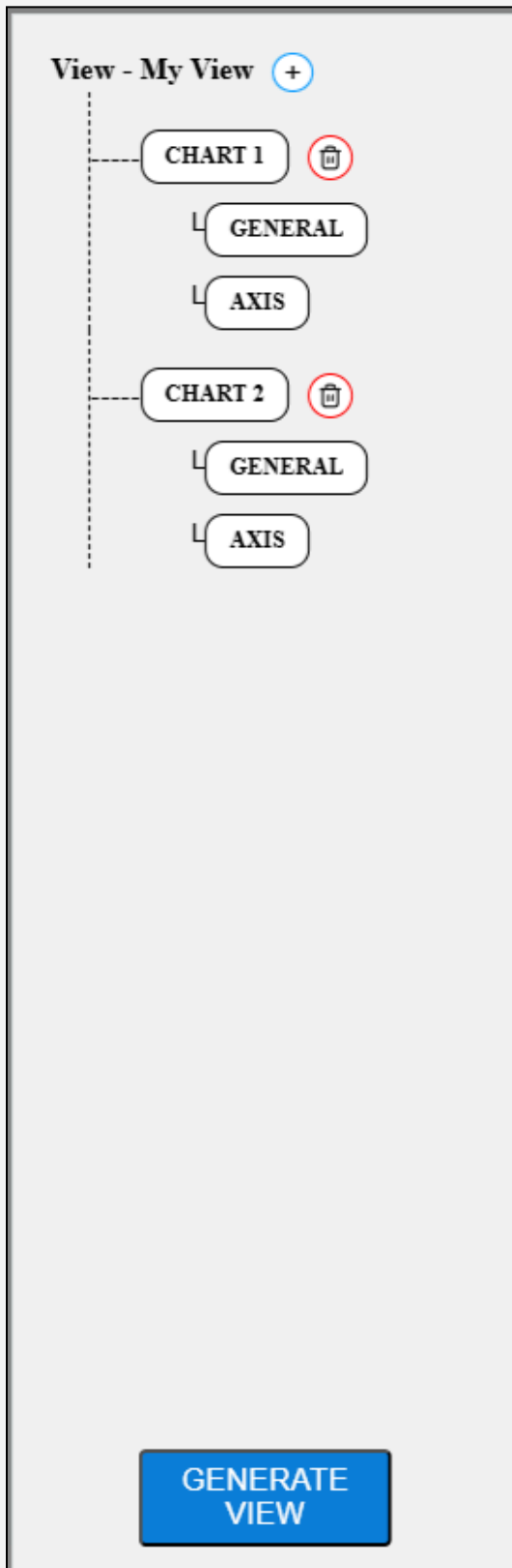
DIRECTION_ANGLE

REMOVE

ADD VAR

Plotting

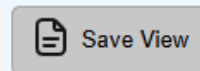
When creating a new view, a side panel will appear in view mode that allows for configuring your charts.



By using this menu, charts can be created, deleted, configured and generated.

(this feature will be completely covered soon in a below section, users wanting to know more about this should scroll down and find “How plotting works in NetSeaDF”)

Saving Views



By clicking on the button in the top bar, users can save their current view to a file, allowing them to generate and / or display their plots later.

Data Transfer

By using the transfer functions in the app, it is possible to construct a view of several plots, include the plot data into the chart when it is saved, and allow others to import the saved view into their installation of NetSeaDF and view the plot without needing the files it took to build the plot in the first place.

(more on this later).

Viewing app notifications and errors

By clicking on the notification button, you can bring up the notifications popup which holds all messages the app has outputted to inform the user of what is going on.

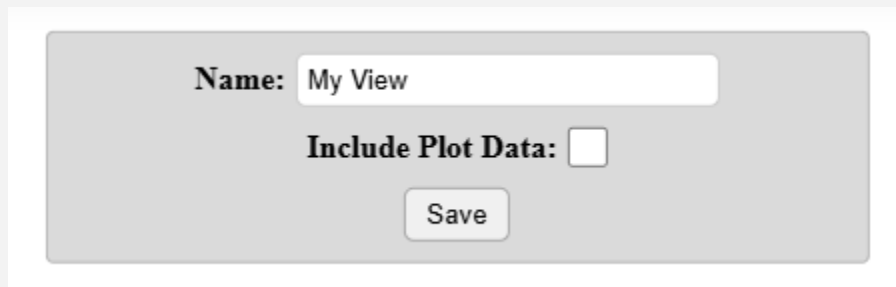


This feature can be used to better understand the status of the app and how it is responding to your actions.

Saving views

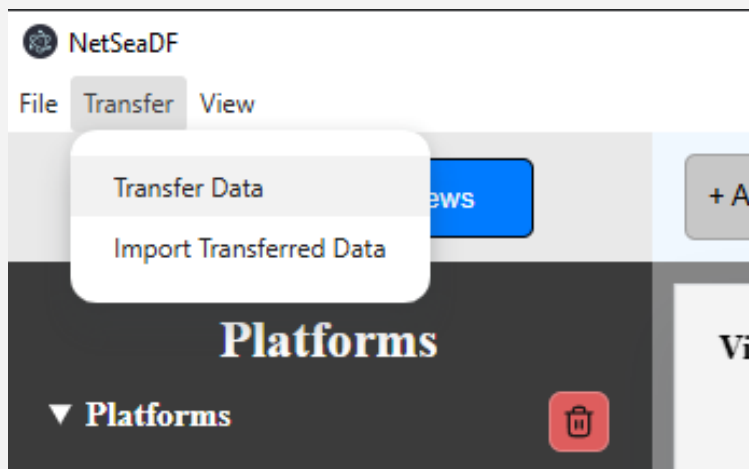
In NetSeaDF, it is possible to take your current view and save it to a file.

This allows you not only to view the file later, but (depending on how it was saved) allows you to view the plotting data again without needing to have the files it took to generate that plot.



Transferring data

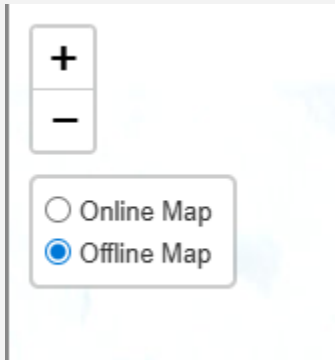
NetSeaDF has a feature built into the app through the window bar (under the Transfer tab) for transferring user data in and out of NetSeaDF installations.



(this page marks the end of the application main features)

Online and Offline Maps

NetSeaDF has the ability to visualize the map without needing the internet. This is a massive advantage if traveling for a conference.



In order to switch between online and offline maps, use the control in the top left of the map.

You can easily tell if the online map is unavailable because it will be blank.

Note: The online map utilizes network requests and sometimes latency and bad IP packets can cause issues with loading the map, this is another reason why the offline map is useful, it will always work.

Note: The offline map does not support full zoom. After a certain point, the offline map will become blank. This should not impede research and is just a small visual discrepancy. By keeping the map not significantly zoomed in, you can guarantee that the map will never go blank.

Note: If the map goes blank and tiles on the map can't be retrieved, the console will show this error.

```
✖ GET lib/leafletMaps/USGS/5/19/4.jpg:1 ⓘ  
file:///D:/Repository/SavV/src/lib/leafletMaps/USGS/5/19/4.jpg  
net::ERR_FILE_NOT_FOUND
```

This is not a big deal, and it should not cause any issues with the map, but it does mean the map tiles can't be retrieved. This issue is not of concern and can be ignored.

How to create a view

By changing to view mode and bringing up the “New View” Screen, you can assemble a plot.

The screenshot shows a 'New View' configuration screen with the following fields and options:

- NAME:** My View
- TYPE:** Line
- DATA:** Currently Selected (dropdown menu is open showing options: Currently Selected, All Visible On Map, All Hidden On Map, All With Notes, All (!), Custom Selection)
- ONLY USE FILES:** (checkbox)
- IGNORE FILES:** (checkbox)
- CREATE VIEW** button

There are multiple different settings you can choose to customize your chart, arguably the most important is the “DATA” setting, since this defines where your data comes from. By choosing “Currently Selected” (the default), the view will include all platform files you have selected. If you choose either of the other two, you are choosing files based on the map filtering.

Note: (Beware of using files with no pressure data, they will be auto ignored from all view creation since they cannot be used for plotting).

Note: (Sound Speed will be available as an option for plotting, but only if every file used in the view has each PRES, PSAL & TEMP. To ensure SSP always is an option, check the setting called, “Ignore Files Without PRES/PSAL/TEMP:”)

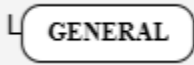
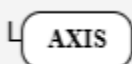
IGNORE FILES WITHOUT PRES/PSAL/TEMP: ☒

How plotting works in NetSeaDF

When creating a new view, a side panel will appear in view mode that allows for configuring the view. All charts and their settings are managed from this side bar menu called the “View Graph”.

The following buttons will control your charts.

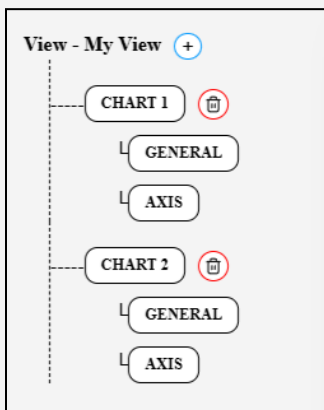


<u>Icon</u>	<u>Name</u>	<u>Description</u>
	Add Chart	Adds a new chart to the view.
	Delete Chart	Deletes the current chart (won't work if there is only one chart).
	Chart Overview	Overview of the current chart and all its settings.
	Chart General Settings	General settings for the current chart. (includes name & other settings)
	Chart Axis Settings	Axis settings for current chart. (this holds the vars being used for the plots).

This side panel is structured such that it is easy to see which components in the sidebar connect to which part of your view.

The top of the panel shows the name of your view, since the view encapsulates everything, all configuration and settings are downstream from there.

Next you have Charts that extend from the view, each of which has a general and axis component resembling that chart's settings.



You can see that by adding an additional chart, the View Graph just further expresses this relationship of Views → Charts → Chart Settings.

Chart Settings

General Settings

Each Chart has its own general settings, the following is an overview of each of the general settings.

<u>Name</u>	<u>Description</u>
Name	The name of the specific chart.
EnableZoom	Whether the chart supports zooming or not.
EnableDataPoints	Whether the charts plots will include points, or just generic lines.
EnableAxisPointers	Whether the chart will include an axisPointer or not. (This is explained later on)

The Axis Settings

Each chart also has an axis settings menu for configuring the axis' of the chart.

Chart 1 Axis Settings

ADD AXIS +

► Axis 1 Y PRES_ADJUSTED Remove Axis

► Axis 2 X SSP Remove Axis

Axis Side

Axis Variable

<u>Name</u>	<u>Description</u>
Axis Side	The axis side you want the axis to be for.
Axis Variable	The variable from the selected .nc files you want to use.

View Generation

After a user has finished configuring their charts, it is time to generate the view.

At this point in the “view configuring process”, the view is entirely composed of data structures, there is no actual visual chart or plot yet. In order to finally display the plots based on the user's view configuration, the view will have to be generated.

GENERATE VIEW

This can be done by clicking the button.

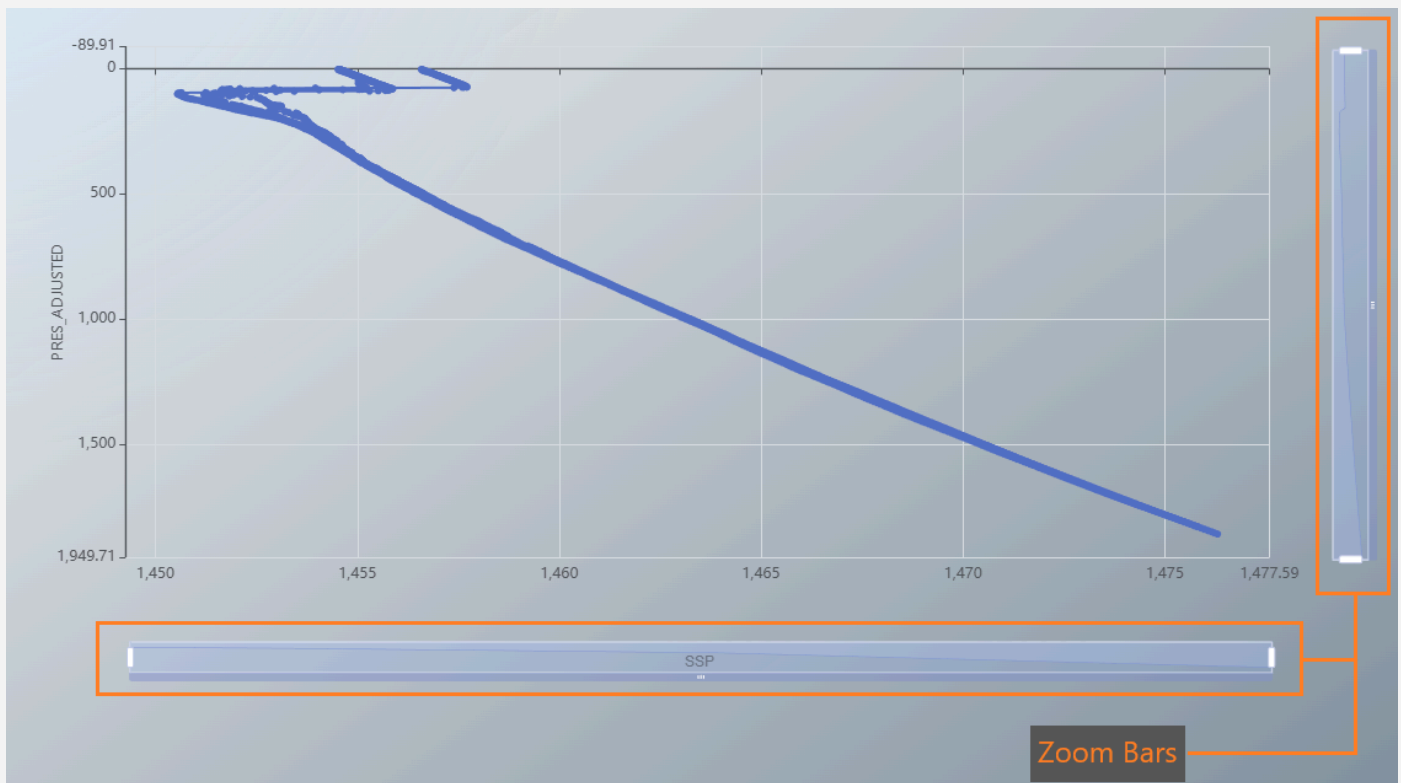
Once this button is clicked, NetSeaDF will begin working in the background to pull the necessary data out of the view-included .nc files and display them based on the settings of each chart and the view.

View Output



After the view has been generated, a new component named “Output” will appear within the view graph. By clicking on the output, you can visualize the result of the view. This is where the user's plots reside.

So long as the user kept enableZoom enabled, the plot will have zoom bars, allowing the user to zoom in to different sections of the plot.

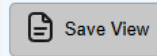


Note: (users can edit chart settings at any time, but some of those edits won't be reflected in the plot until the view is regenerated)

View Saving

Once a view has been visualized, the user will need to make the decision as to whether to save the view or not. NetSeaDF has full support for saving views to files for viewing later.

Once a view has been explored and is ready to be saved, click the



button in the top bar to save the current view.

A screenshot of a "Save View" popup window. The title bar says "Save View". Inside, there is a text input field labeled "Name:" with the text "view name" inside. Below that is a checkbox labeled "Include Plot Data:". At the bottom is a "Save" button.

Save View

Name:

Include Plot Data: ☐

Save

After clicking this button, a popup will appear, asking the user to finalize saving the view.

After clicking the save button, the view will be saved as a *.netseadview* file.

The “Include Plot Data” Option

Within the Save View popup, the user is presented with an “Include Plot Data” checkbox. This option determines whether the generated plotting data is written directly into the saved view file or not. When enabled for a generated view, the file stores the plot output along with the view configuration, allowing the user to reopen that view later with its plots already preserved, without needing to regenerate them from scratch.

This is one of the most valuable features in NetSeaDF because it allows saved views to remain useful even when the original dataset files are no longer available. The tradeoff is that view files with embedded plot data can become several megabytes larger, but in return they can be reopened instantly with their plotting results intact. This makes saved views far more portable, reliable, and convenient for sharing or long-term reuse.

It is also important to understand the limitations of this behavior. A saved view with preserved plot data can still be opened and inspected without the original source files, but it cannot be regenerated unless all of the files originally used to build that view are still available in the app. In other words, enabling Include Plot Data preserves the generated output, not the full ability to rebuild the view from missing datasets.

In-App Notifications

NetSeaDF has a feature built in for notifying the user of different events and alerts that occur during the software's use. This is to ensure that if any abnormal behavior or important event happens within the app, The user is promptly notified and can react accordingly.

The main purpose of this system is to not only notify the user of how their session might be impacted due to how they are using the software, but also to give visual feedback if something goes wrong.

Some notifications will cause a popup, some of them won't.

Each notification has a message attached to it to inform the user, and each message has varying degrees of seriousness depending on what the notification is informing the user about.

Therefore, each notification within NetSeaDF is designated with a specific type of seriousness level. The seriousness of any notification that could occur, fits into 3 main categories.

- Info
- Warnings
- Errors

These 3 classifications are further broken down below.

Icon	Name	Purpose
	Info	Notifies the user of basic updates and information as they use the app. It involves things that might be helpful to know but usually are not that important and certainly are not things that require reading to continue using the app. An example would be "Imports Complete".
	Warning	Notifies the user of concerning situations the user might be getting themselves into as well as potential issues that might arise if they continue. An example would be "No Files To Plot".
	Error	Notifies the user of critical issues with the application and its computations. It involves things that severely impact the apps performance and might make the app incapable of working properly. This notification is very rare and should never be seen. (hopefully)

Logs

There is a logging system built into NetSeaDF that is connected to the notification system. Although some notifications will have a popup and some won't, all notifications create messages in the sessions log, that is true so long as the user has Debug mode enabled in the app. This is an important note about the log system. NetSeaDF is initialized with new installations to have debug mode off by default. To change this, go into the user settings, enter the "DEBUG" section, and enable the setting called "enableDebug".

So long as enableDebug is enabled, each time you reboot/start the application, the app will auto create a log file that it writes to and will continue to write to for the duration of the app's running session.

This log is written with a custom file format extension called .netseadflog

This file format is a simple, plain-text format that is easily readable in a text editor like VSCode.

Example of a log entry,

```
05-23 5:52:54 PM | map.html-1357 [S:1] [C:INPUT] [ID:0002]
| Timeline Filter Unset
| Not all required values have been set to filter the timeline!
```

These logs are primarily meant for the developer of the app. But if you want to understand the structure of these logs, the following will provide context.

```
05-23 5:52:54 PM | map.html-1357 [S:1] [C:INPUT] [ID:0002]
| Timeline Filter Unset
| Not all required values have been set to filter the timeline!
```

Color	Meaning
Green	Timestamp
LightBlue	File and Code Line where the log originated from
Yellow	Seriousness level (0 = info, 1 = warning, 2 = error)
Orange	The notification class.
Pink	Log ID number (this can be ignored as it was never implemented in NetSeaDF)
LightTeal	Title of the log message
Lavender	The actual log message

Log Classifications

There are many different log messages that can appear in the .netseadflog files but they don't always originate from the same areas of the application.

In order to bring more clarity to log messages, the log classification system was developed. Every log message has a classification that describes the type of action that the surrounding code was performing leading up to the log.

It is designed to make sifting through logs easier by identifying each log message by their classification. By looking at the class of a log message, it makes it easier to understand what the application was doing that led up to the message being added to the log.

The following is a list of the main log classes, what their classification means and an example of what circumstances have to exist for a log of the particular class to be logged.

Name of class	Description	Example
INPUT	Represents log entries that involve user input, you would see this log typically if you are clicking buttons.	Clicking the import button
FS	Stands for FileSystem and involves logging events with importing, exporting, creating and removing files.	A file being imported into the app.
CALC	Stands for Calculation and involves logging events where complex math is being performed.	Calculating timestamp differences for underIceFlags.
MAP	Represents events where the map changes, this includes things like adding/removing map markers.	A platform being added to the map after import.
VIEW	Represents log events pertaining to the current view. This includes creating a view, as well as reloading a saved view.	Creating a new view.
PLOT	Represents log events involving the construction and viewing of plots. This involves things like building the data for plots.	A plot finishing its generation.
STATE	Represents changes to the applications state. Not every change to state is logged, only the important ones.	(any change to app session data)
QUEUE	Presents log events that pertain to import and removal queues. This involves importing datasets and removing files.	Importing more than one file at a time.
ELECTRON	Represents logging events made by the system at a deeper level. (electron is the framework that controls the apps window)	Clicking the Log AppState Option in the windows tools tab.

It is important to note that each log class has a respective log class setting in the settings menu that allows for disabling or enabling a specific class of log. This makes it possible to enable logging, but only for inputs and file system events, allowing for finer control over what needs to be logged for debugging.

Note: (By default, all log classes are enabled, meaning when enabling EnableDebug, any log event that happens will show up in the log. Disable these class types when you want to maintain control over logs.)

How to transfer user data between NetSeaDF installations

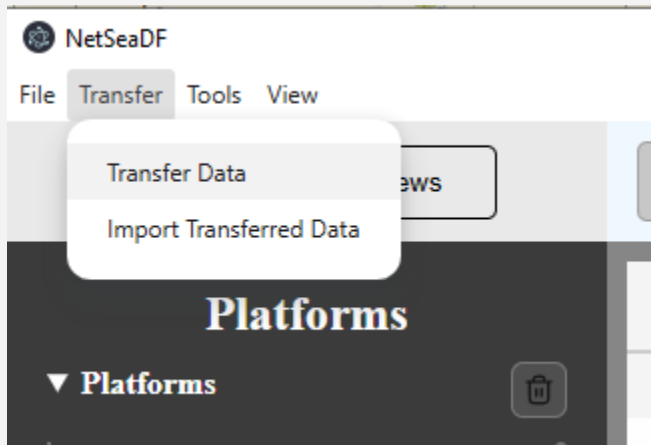
In NetSeaDF, "transferring data" refers to the process of exporting and importing collections of user data including platform files (NetCDF datasets), saved views, application settings, and filtered variable lists between different installations.

Key points:

- The transfer feature packages selected data (platforms, views, settings, filtered variables) into a single archive file.
- Users can export this archive for backup, sharing, or migration to another computer.
- On import, the archive is unpacked and its contents are integrated into the receiving app's data folders.
- The process supports overwriting existing files (if enabled in settings) and ensures all related metadata and configuration are preserved.g.

This feature streamlines collaboration, backup, and deployment of user-specific data and configurations across different NetSeaDF environments.

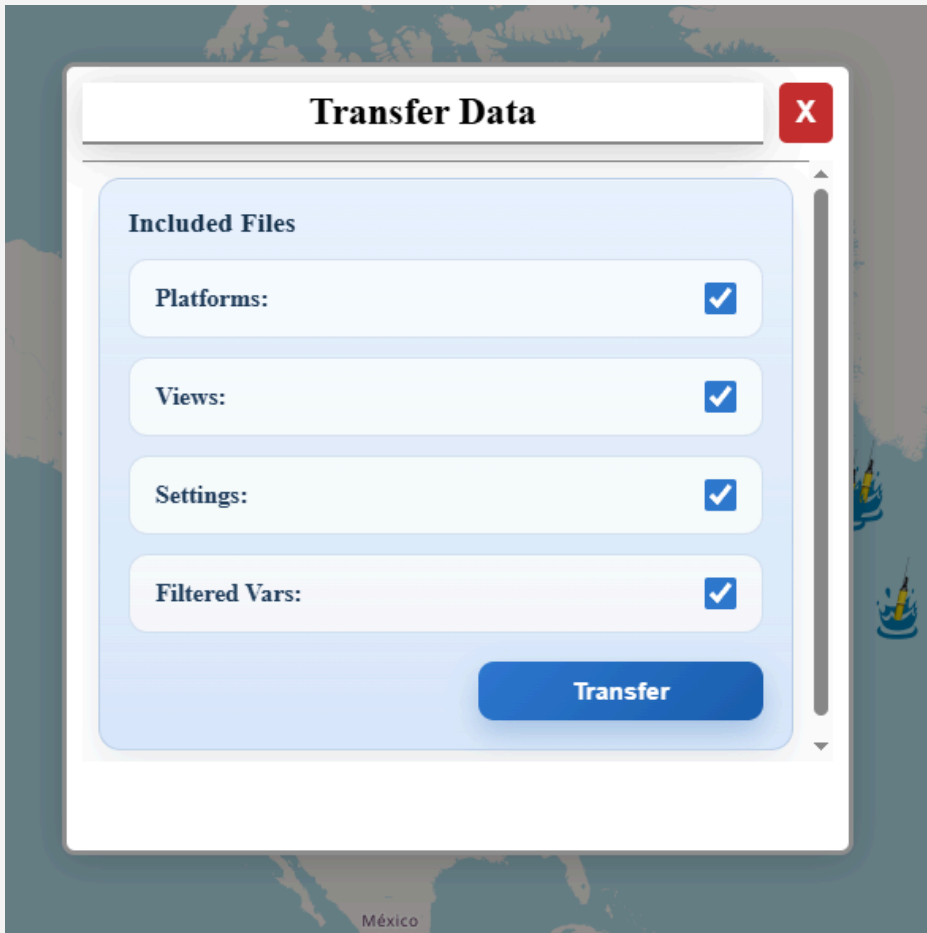
Exporting transferred data



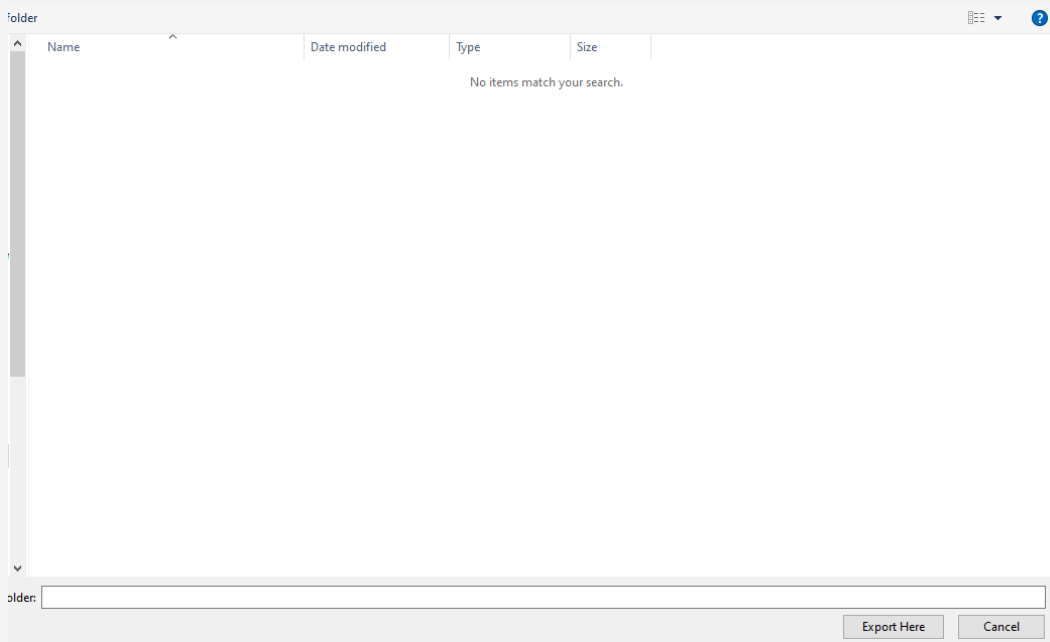
By clicking on the transfer tab in the window bar, users can transfer data.

- **Transfer Data**
 - Used to export data
- **Import Transferred Data**
 - Used to import transferred data

After clicking the transfer data option, users will be presented with a popup, allowing them to decide what they want to transfer.



- **Platforms**
 - .nc Files
- **Views**
 - .netseadfview files
- **Settings**
 - User settings
- **Filtered Vars**
 - List of filtered vars

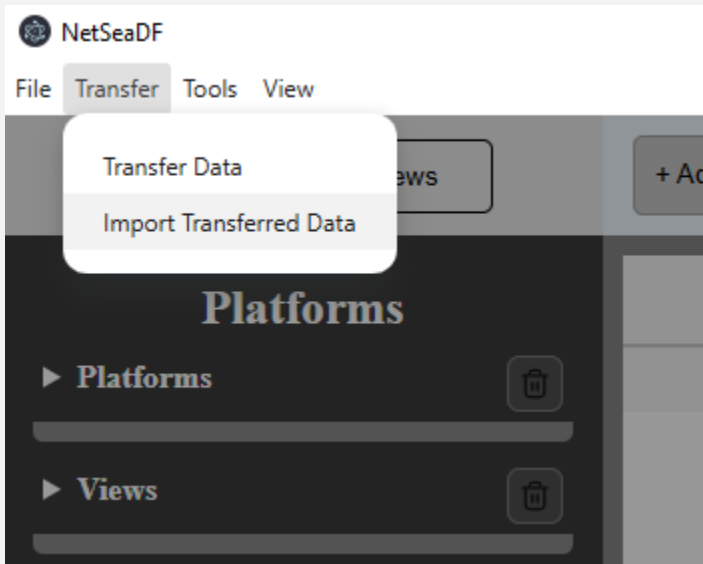


After clicking the transfer button, a popup will appear on your computer to decide where to put the transfer package.

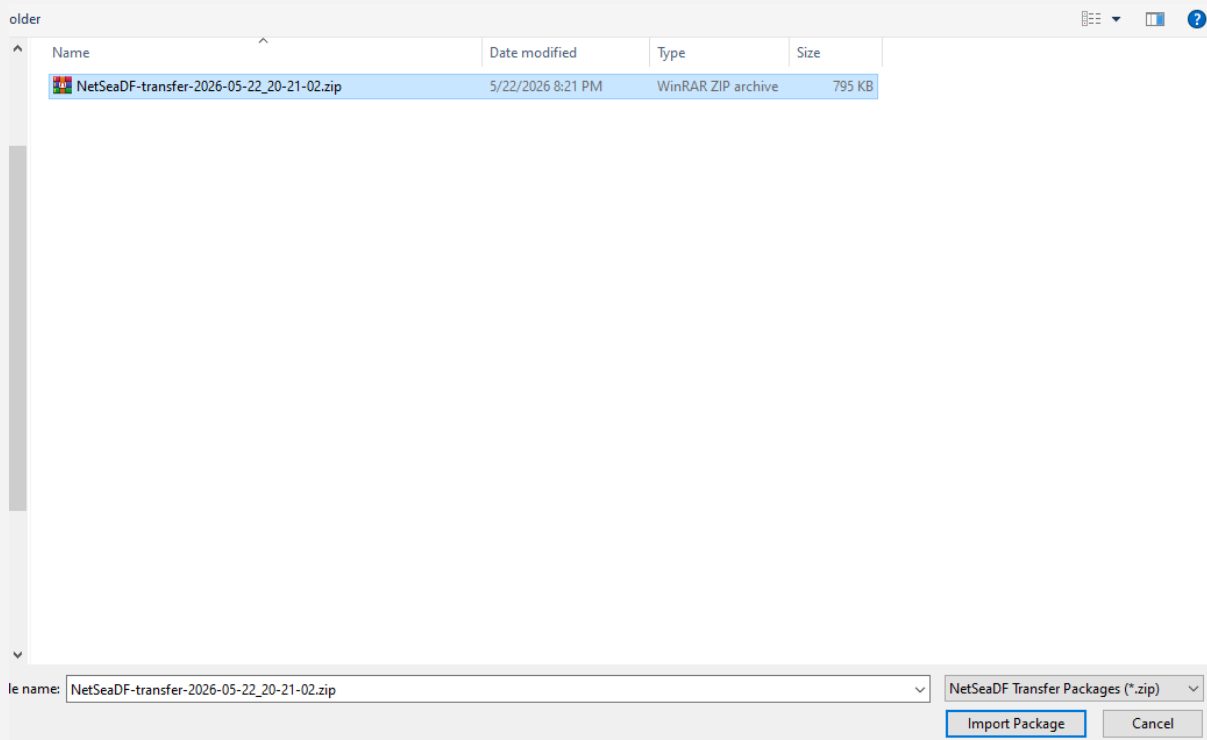
Click “Export Here” to finalize the operation.

Importing transferred data

By clicking the “Import transferred Data” option, a popup will appear on your computer to select the specific transfer package to import.



The transfer packages should start with the name “NetSeaDF-transfer” (unless a user changed it). After clicking on the package, click “Import Package” to finalize the operation.



By this point, all of the data that was transferred should be automatically imported into your installation!

User Settings

The following is a list of the most common settings in NetSeaDF.

Section: debug

- **enableDebug**: Turns on debug mode for the application, enabling extra logging and debug features.
- **DebugFS**: Enables debug logging for file system operations.
- **DebugCalc**: Enables debug logging for math.
- **DebugInput**: Enables debug logging for user input actions.
- **DebugState**: Enables debug logging for application state changes.
- **DebugQueues**: Enables debug logging for queue operations (import/remove).
- **DebugView**: Enables debug logging for view interaction.
- **DebugMap**: Enables debug logging for map-related actions.
- **DebugPlot**: Enables debug logging plotting.
- **DebugELECTRON**: Enables debug logging for electron events.
- **onlyLogSeriousNotifications**: If enabled, only logs notifications marked as serious (suppresses less important logs).

Section: basics

- **mapDefaultZoomLevel**: Sets the default zoom level for the map when the app loads.
- **MaxTimeTillUnderIceFlag**: Sets the maximum time (in days) before a platform is flagged as potentially under ice.
- **sidebarListFontSize**: Sets the font size for items in the sidebar list (integer, range 10–20).
- **HideBlockingPopupForClickedViews**: If enabled, suppresses blocking popups when a view is clicked.

Section: IO

- **enableImportOverwriting**: Allows imported platform files to overwrite existing ones.
- **EnableImportOverwrittingForTransferredPlatforms**: Allows transferred platform files (via data transfer/import) to overwrite existing ones.
- **EnableImportOverwrittingForTransferredViews**: Allows transferred view files (via data transfer/import) to overwrite existing ones.
- **enableSaveOverwritingForViews**: Allows saving a view to overwrite an existing view file.

Section: views

- **HideQCVarsForAxisDropdown**: Hides variables from axis selection ending in “QC”.
- **HideDMVarsForAxisDropdown**: Hides variables from axis selection ending in “DM”.
- **HideERRORVarsForAxisDropdown**: Hides variables from axis selection ending in “ERROR”.

Troubleshooting

Considerable effort has been made to ensure NetSeaDF does not run into issues, but there is still a chance something might go wrong. In the event of an issue, these are the 3 recommended ways of overcoming those issues.

Help: Rebooting

The app comes with a built-in feature for performing the equivalent of a “turn it off and back on again” action. This can be done by going to the window bar and clicking “view”, then “reload”.

This is by far the easiest, most user friendly option and will resolve most issues.

Help: Notifications

The notification system of NetSeaDF acts as the most user-facing way to help oneself. If something goes wrong in the app.

If there is an issue, the notification menu in the app as well as popups on screen should inform the user of exactly what is going on. Paying attention to these 2 features in the app will ensure the user stays informed.

Help: Logging

The logging system is the next feature that is user facing, albeit a bit less, but still invaluable if something has gone wrong.

The logs represent a feature that is deeper and more “data-like” compared to the popups and notifications in the app, but by looking at the logs, users can understand just about everything that happens each step of the way in the app.

It is recommended that users try and conceptualize which log class their issue might under and only enable debugging for that specific class. For example, if a button is not working, only enabling debug and DebugInput will allow logs to be generated for testing, but only input class events will be saved to the log.

Help: Tooling

At the top of NetSeaDF's window bar, there is a tab called "Tools". By selecting this tab, the user will be presented with a multitude of actions the user can perform.

The following is a list of each of these options in the tools tab.

- **Clean Up References**

- This action will traverse NetSeaDFs database and find any discrepancies between the apps data and the imported .nc files.
- .nc files that have not synced with the applications database properly will cause issues if those files are interacted with in the app. It is rare for this to happen, but it occasionally might. An .nc file should never exist in NetSeaDF if it is not properly connected to the database.
- The benefit with this option is you can click it whenever you want, as many times as you want. It is very friendly to the app and does not have to be used sparingly as only desynced files will be deleted, all other data will be intact.

- **Log AppState**

- AppState is a variable in the application that holds all user data that persists throughout a session. It holds the data for what files you have selected, whether a user is in map mode or view mode, the user's current plotting data, map data, map marker data. Nearly everything the app needs per session to run properly.
- Because it is the central hub for session data, issues in the app can often be identified by the status of this gigantic data structure.
- This data structure (AppState) can be dumped out of the app into the log and console for debugging. This will most likely not be useful for regular users, but to the developer of NetSeaDF, this is invaluable.
- If the developer of the app is helping you with an issue, give him the output of this variable.

- **Print PATH**

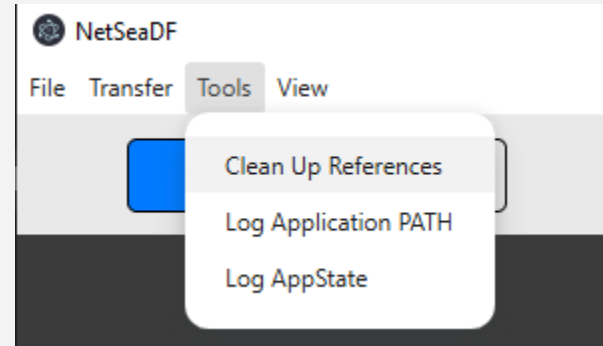
- The application exists somewhere in the user's computer, this option will print the path of the app so users can easily find where the app exists within their computer.

Common Issues

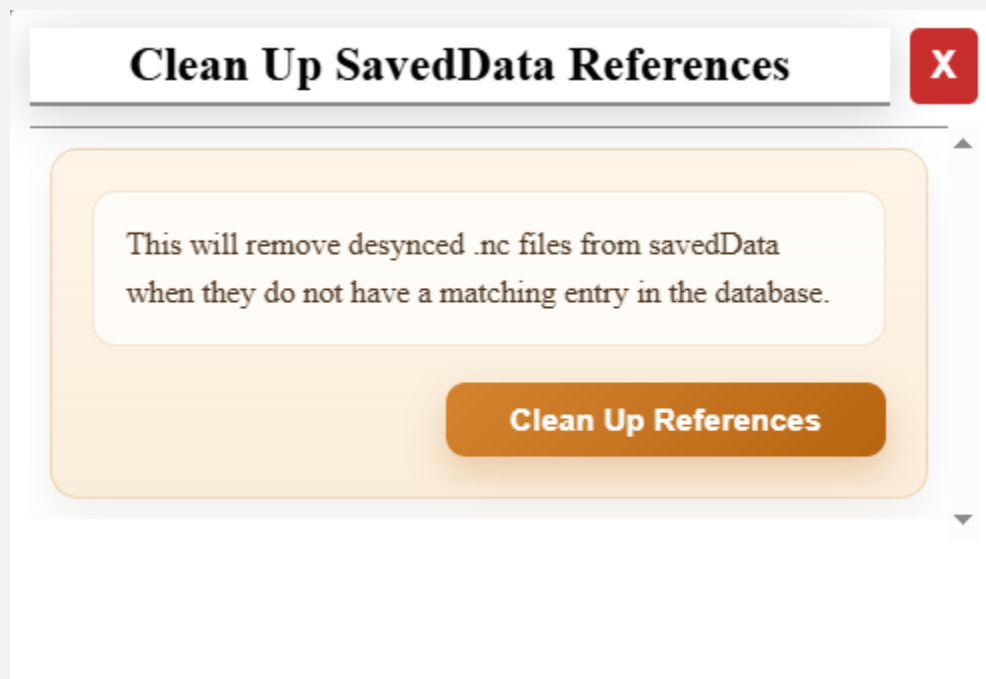
File Desync

A file desync issue happens when an .nc file still exists in the apps folder but its matching reference is missing from the app's internal database, which can cause the file to appear broken, load incorrectly, or behave inconsistently in the sidebar and views.

The simplest fix is to use the "Update References" button in the Tools tab located in the window bar, which scans for these mismatches and removes desynced file entries that no longer have a valid reference connection. After running it, the app's list of saved files and the database are brought back into sync, which helps restore normal behavior and prevents desynced .nc files from lingering.



Note: It is easy to tell when an .nc file has become desynced because the name of the file will be red.



Developer Section

Software used in development

Name	Icon	Purpose
Microsoft 3D Paint		All visual editing to pictures and icons. Rough sketchings of designs.
VSCode		Text editor for all software engineering.
GitHub Desktop		Version Control System, supplied the repository for the application.
Draw.io		All diagrams and wireframes.

Tech Stack

The following is the full tech stack of NetSeaDF.

- **Programming Languages**
 - Python3
 - JavaScript
- **Data formats used**
 - .yaml
 - .json
- **Database Architecture**
 - JSON
- **FRONTEND**
 - [leaflet.js](#) = 1.9.4
 - echarts = 6.0.0
- **BACKEND**
 - [node.js](#) = 24.11.1
 - electron = 40.0.0
 - electron-builder = 26.8.1
 - adm-zip = 0.5.17
 - archiver = 8.0.0
 - numpy = 2.2.6
 - packaging = 26.0
 - pandas = 2.2.3
 - pip = 24.3.1
 - python-dateutil = 2.9.0.post0
 - six = 1.17.0
 - tzdata = 2025.3
 - xarray = 2024.11.0
 - gsw = 3.6.19
 - netcdf4 = 1.7.2
 - fs = 2.4.16
 - cftime = 1.6.4

Tech Stack Explanation

Although NetSeaDF uses many packages across both the Python and JS ecosystem, the application truly runs on the packages below,

- Electron (managing the applications window)
- [node.js](#) (provides the engine for JS)
- Leaflet (runs the applications maps)
- Echarts (handles all charts and plotting)
- Xarray (handles all .nc file parsing)
- Numpy (handles many complex python data structures and formats)
- Gsw (solely responsible for providing the equation for sound speed)

These packages do 90% of the work.

JavaScript is also making heavy use of built in node modules.

Most importantly,

- Node:child_process (makes python+JS communication possible)
- Node:path
- Node:fs
- Node:os

SDL (Software Development Lifecycle)

Throughout the lifecycle of NetSeaDF, a highly structured life cycle was utilized to ensure stable feature releases with new software versions.

The SDL of the application revolved around the idea of features. Progress was made in relation to features that were expected to be in the application. Therefore, each stage of the SDL is a further increment in progress in releasing a new feature in the application.

The following is a detailed explanation of each stage of the SDL for NetSeaDF.

Stage: Expectations

The first stage involved communication with others to figure out what exactly is needed within the application. This stage involves holding dialogue about features that are expected to be added or considered helpful to have in the application. This dialogue is very casual and abstract.

This stage in some sense, is always ongoing. But it is always the prerequisite to designing something. By the end of this stage, all individuals should have a mutual understanding of what the bigger picture end result should look like.

Stage: Planning

At this stage, conversation should be at the level of an individual feature. Communication should be resulting in “I will work on adding this feature next”.

This stage should result in everyone involved understanding what feature is going to be focused on next. The end of this stage should narrow things down such that there is mutual understanding of where development resources are being allocated as work continues.

Stage: Design

At this stage. There is full technical conversation taking place about what this new feature in the works should look like, how it should function, what it needs to be able to accomplish and how long it will take to complete.

By the end of this stage, communication should lead to a mutual understanding of what is going to be implemented, the extent of its abilities, and how long until it is ready to be used by an end user.

As communication at this stage wraps up, all individuals involved should have an expectation to see a finished feature by the expected time of completion.

Stage: Architecting

This stage can be thought of as technical design from a computation and programmatic level.

At this stage, developers should be creating diagrams, making notes and organizing themselves to come to a conclusion of how this new feature will be represented within code, what data structures will be utilized to make the feature function properly, and what design patterns (if any) are going to be used to maintain this feature over time.

This stage is about performance, technical debt and algorithms.

This is the stage where an understanding is established of how the code will be written to achieve a stable, high quality feature.

Stage: Implementation

This stage involves taking the architectural designs of the feature and actually programming the feature into the codebase, fixing bugs as they popup.

Developers should pay close attention to the time it is taking to implement the feature, making sure they hit the expected date of completion on time.

Stage: Pre-Production

By this stage, the feature is fully implemented in the app and is in the process of being tested.

As long as the feature has been extensively tested and no bugs remain unfixed, the feature is ready for release to the end user.

(it is usually good to check in with others again to ensure what has been built matches expectations)

Stage: Production

By this point, the feature is ready for production. The next time a new release is published, this feature will be in the release notes.

(by the end of the production stage, people should reconvene, re-establish expectations for the project as a whole, and begin planning the next feature).

map.html (The most important file)

map.html is the main file holding the interface for NetSeaDF. It serves as the fundamental component of the app and in some sense can be thought of as the “screen”.

Note: Don't be confused by the name “map”. This file is not just holding the map mode, it is holding the view mode as well. The decision to call it “map” was an early mistake and after thousands of lines of code, renaming this critical file would result in disaster. So the original name remains.

Key Features and Structure:

UI Layout:

- Header/Banner: File import, view creation, save, delete, and settings buttons.
- Sidebar: Lists available platforms (datasets) and saved views, with collapsible sections.
- Main Area:
 - Map mode: Interactive map (Leaflet-based) showing dataset markers, timeline controls, and filtering options.
 - View mode: For creating and managing custom data views and charts.
- Popups: Modular popup system for notifications, settings, filtered variables, data transfer, and cleanup.

Styling and Libraries:

- Uses multiple CSS files for modular styling (sidebar, map, popups, etc.).
- Integrates Leaflet (map) and ECharts (charting).

Modular JS Integration:

- Loads all core modules (DOM, logging, userInteractions, charts, fileHandle, queue, transfer, popup, etc.) via require.
- Centralized the appState object (and is where it lives) and tracks all UI and data state, including selected files, map state, filters, popups, and the users currently existing view.

Map Functionality:

- Initializes a Leaflet map with both online and offline tile layers.
- Loads and displays dataset markers from saved data, with popups showing dataset info.
- Supports marker clustering, focus, and timeline-based filtering (slider/input, 12hr/24hr clock).
- Location-based filtering with bounding box selection and map rectangle overlay.

Data and View Management:

- File import (multi-file, queue-based) as well as deletion.
- Creation, configuration, and saving of custom data views.
- Persistent storage and retrieval of filtered axis variables and user preferences.

Popups and Notifications:

- Popup system for user feedback, notifications, settings, filtered variable management, data transfer, and cleanup of desynced references.
- Logging and error handling are tightly integrated, with logs written to file and/or displayed in the UI.

Event Handling and User Interactions:

- Extensive use of event listeners for all major UI actions (file import, view creation, filtering, navigation, etc.).
- All user actions are routed through modular handler functions, ensuring separation of concerns and testability.

Integration with Python Backend:

- While not directly visible in map.html, the UI is tightly coupled with a persistent Python process which handles scientific data access, computation, and file operations via a JSON-over-stdin/stdout protocol.

Role in the Application:

map.html is the central user-facing component of NetSeaDF, providing a seamless bridge between scientific data, interactive visualization, and user-driven analysis. It leverages a modular, state-driven architecture to ensure maintainability, extensibility, and robust integration with both the JavaScript frontend and Python backend.

It all exists elsewhere, but it all gets connected to here in the map.html file.

Code snippet from map.html showing how all the modules connect to the file.

```
<!-- MODULES -->
<script>
  const path = require('path');
  const fs = require('fs');
  const { ipcRenderer } = require('electron');

  const logging = require('./common/logging.js');
  const unitTest = require('./common/unitTest.js');
  const pathDep = require('./common/pathDep.js');
  const integrations = require('./common/integrations.js');
  const fileHandle = require('./common/fileHandle.js');
  const queue = require('./common/queue.js');
  const transfer = require('./common/transfer.js');
  const DOM = require('./common/DOM.js');
  const userInteractions = require('./common/userInteractions.js');
  const basicFunctions = require('./common/basicFunctions.js');
  const charts = require('./common/charts.js');
  const config = require('./common/config.js');

  const popup = require('./common/popup.js');
</script>
```

CI/CD Workflow

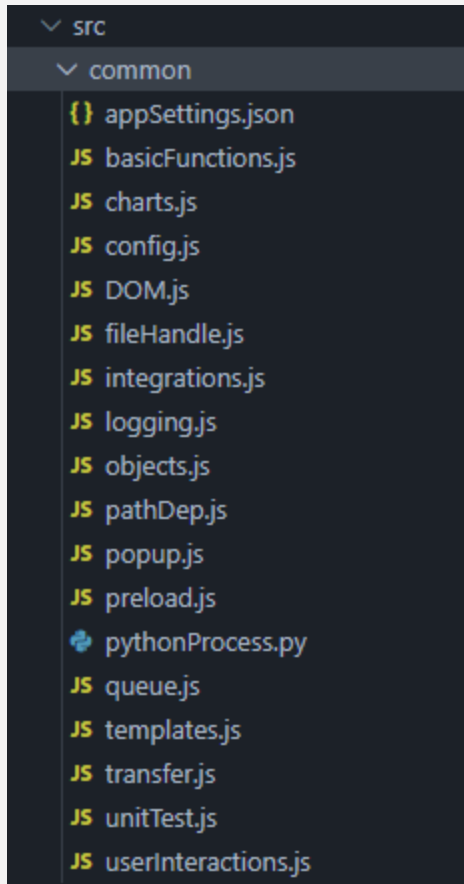
This project uses GitHub Actions to automatically build and release the application on github. The automated workflow runs when code is pushed to the main branch of the repository. This helps make sure the project can still be built correctly as changes are made.

The build process is split into two jobs: one for Windows and one for macOS. Each job checks out the code, installs Node.js 20, installs the project dependencies, and then runs the correct build command. The macOS job also does extra setup for the bundled Python environments, including fixing permissions and installing packages from requirements.txt. When each build finishes, the workflow saves the generated installer file to be included in the next release. For this project, that means a .exe file for Windows and a .dmg file for macOS.

The commands used to create these installers are defined in package.json. They use Electron Builder to package the app, while the detailed packaging settings are stored in electron-builder.yml. That file tells the build system where to place the output and which extra files should be included, such as the Python runtimes, config files, saved data, logs, and queue files.

It's important to understand that this process is derived from a script in the codebase called build.yaml. Because this script is in the codebase, it is in the repository and github recognizes the file and uses its contents to run commands in github. The commands use computers owned by github to take the codebase, and install all dependencies needed, then take the entire codebase and package it into an executable, then make that executable available to be added to each software release. This is how software releases are carried out. The entire process is automated. Remember, NetSeaDF is an actual desktop application, but it only ever exists as code when developing. You need to package the codebase into an executable for a platform, and a computer must be used for this. This project is using an automated system in github to package the code instead of the developer of NetSeaDF needing to do it himself through commands.

Modules



NetSeaDF is over 5,000+ lines of code, but it is not just in one file, it is distributed across several files, each of which is meant to handle a specific responsibility within the app. These files are mini-modules that maintain responsibility over a portion of the application.

The following is a graphic of each module that exists in NetSeaDF. (this is taken directly from VSCode)

At a high level, the common folder is the shared “engine room” of the app. The UI page in map.html calls these modules to read data, manage files, draw maps/charts, show popups, run Python analysis, and handle user actions. Think of this folder as the reusable backend logic for the renderer.

Config.js

NetSeaDF keeps user settings in the config directory. These settings are split across files like basics.yaml, debug.yaml, IO.yaml, and view.yaml. Without a shared config layer, each part of the app would need to manually parse YAML, read files, and handle updates itself, which would create duplication and inconsistency.

config.js solves that by acting as a centralized settings service. It loads config files, parses their key-value data, and caches the result in memory so repeated reads are fast. It provides simple get() and set() access so other modules can read and update settings without worrying about file format details. When values are changed, it writes those updates back to disk so settings persist across sessions.

It also supports importing settings from external YAML files during transfer operations. This makes config.js the single place where configuration reading, writing, caching, and import behavior are handled, which helps prevent settings drift and file-handling bugs.

pathDep.js

pathDep.js is the module that keeps file paths consistent between development and production builds. The following is a detailed list of each type of data in NetSeaDF and where it is stored between source and production builds.

- Source code
 - .nc files (stored in /savedData)
 - .netseadfview files (stored in /savedData)
 - .netseadflog files (/logs)
 - Config files (user settings) (stored in /config)
- In Production
 - .nc files (stored in /Dist/savedData)
 - .netseadfview files (stored in /Dist/savedData)
 - .netseadflog files (/Dist/logs)
 - Config files (user settings) (stored in /Dist/config)

As you can see, where data is stored depends on whether the app is run in VSCode for example, or as an executable.

In development, app data is stored directly in top-level folders such as savedData, logs, and config. But in production, the app's data is stored inside dist (for example, Dist/savedData, Dist/logs, and Dist/config).

This includes .nc files, .netseadfview files, .netseadflog files, user config files, and queue files.

This difference exists so end users have one clear point of entry for all application data and is supposed to make it easier to find what you need since it's all in the Dist folder. However, this design decision also creates a path mismatch problem: the same resource can live in different locations depending on whether the app is running from source or as a packaged executable. Therefore, the app must know whether itself is running as an executable or as regular source code so that the app can adjust file paths accordingly to ensure they won't break.

The code that ensures paths never break and can always resolve properly is all within pathDep.js, this is its sole purpose.

Centralizes all path resolution logic for the app, providing robust, environment-aware utilities for constructing absolute and relative paths to key data, config, and queue files and folders. Ensures that all file operations work seamlessly in both development and production (packaged) environments.

Key Responsibilities:

- Path Resolution: Dynamically resolves paths to core folders (logs, savedData, config, queues) based on the current environment (dev or production), using Electron's resource path when packaged.
- Folder Level Mapping: Maintains a map of folder names to their sublevel depth from the project root, enabling correct relative path construction.
- Utility Functions: Provides helpers to get the current folder name/path, compute the path from the current directory to the root, and validate folder names.
- Constants: Exports precomputed absolute paths to all critical files and folders (e.g., simpleData.json, filteredVars.json, import/remove queue files), so all modules can reference them reliably.

How It Works:

- Uses the current `__dirname` and a folder name to compute the correct path, adjusting for dev/prod mode.
- Exports both utility functions and constants for use throughout the codebase.
- Handles errors and invalid folder names gracefully, logging issues for debugging.

Typical Usage:

- Imported by all modules that need to read, write, or check files in the app's data, config, or queue directories. Ensures that file operations are always pointed at the correct location, regardless of how or where the app is run.

Notable Features:

- Environment-aware: works out-of-the-box in both development and packaged production builds.
- Centralizes all path logic, reducing duplication and risk of path errors.
- Extensible for new folders or files as the app grows.

fileHandle.js

Centralizes all file and metadata management for the app, handling data files, view files, and the core simpleData.json index. This module provides robust, persistent, and extensible infrastructure for saving, loading, importing, exporting, and deleting all user and app data.

Key Responsibilities:

- **File Existence & Copying:** Checks for file existence, copies files (with overwrite logic), and saves buffers as files in the correct directories.
- **Data Listing & Filtering:** Lists saved data and view files, with extension filtering and directory management.
- **simpleData.json Management:** Loads, parses, and updates the main metadata index for all datasets, including entry existence checks, key/value access, and full entry retrieval.
- **Data Import/Export:** Handles batch import and removal of files via persistent queues, integrating with the Python backend and UI for robust, resumable workflows.
- **Data Deletion:** Deletes both data files and their metadata entries, ensuring consistency and cleanup.
- **Dataset Serialization:** Saves new datasets to simpleData.json, pairing coordinates, storing attributes, and updating in-memory state.
- **View Serialization:** Serializes/deserializes view objects for saving/loading, with schema/versioning and support for plot data caching.
- **View File Management:** Saves and loads view files, with safe naming, overwrite protection, and error handling.
- **Directory & Name Utilities:** Ensures required directories exist and sanitizes view file names for safe storage.

How It Works:

- Reads and writes all data and view files, as well as the simpleData.json index, using robust error handling and in-memory caching.
- Integrates with the queue system for batch import/remove, and with the Python backend for data extraction and validation.
- Provides modular, reusable functions for all file and metadata operations, supporting both interactive and batch workflows.

Typical Usage:

- Used by all major app modules for any operation involving data or view files, including import/export, deletion, metadata access, and UI updates.

Notable Features:

- Persistent, robust management of all user and app data.
- Tight integration with import/export, queue, and Python processing infrastructure.
- Extensible serialization/deserialization for future data and view formats.
- Comprehensive error handling and state synchronization for reliability.

queue.js

Manages persistent import and remove queues for batch file operations in the app. This module enables robust, resumable, and trackable processing of file imports and deletions, supporting workflows like bulk data migration, restore, and cleanup.

Key Responsibilities:

- Import Queue Management: Reads, writes, and updates the import queue (stored as JSON), allowing files to be scheduled for import, marked as done, or cleared.
- Remove Queue Management: Handles a separate queue for file deletions, supporting batch removal, status tracking, and queue clearing.
- Queue Entry Handling: Adds new entries to queues with metadata (file name, destination, status, overwrite flag), and removes entries as they are processed.
- Persistence: All queue operations are file-based, ensuring that queued actions survive app restarts and can be resumed or retried safely.

How It Works:

- Each queue is a JSON file on disk (paths managed by pathDep).
- Functions are provided to read, write, add, mark as done, and clear both import and remove queues.
- Entries are objects with relevant metadata for processing and status tracking.

Typical Usage:

- Used by backend and integration logic to manage batch file imports (e.g., from transfer packages) and deletions, ensuring that all operations are tracked, recoverable, and can be retried or resumed as needed.

Notable Features:

- Fully persistent, file-based queue system for reliability.
- Supports both import and remove workflows with clear, modular functions.
- Enables robust, user-friendly batch operations and error recovery.

transfer.js

Handles the packaging, export, import, and validation of data transfer packages in the app. This module enables users to bundle, share, and restore platforms, views, settings, and filtered variables as portable .zip archives.

Key Responsibilities:

- **Transfer Options & File Collection:** Normalizes user-selected transfer options and collects the relevant files (platforms, views, settings, filtered variables) from the app's data directories.
- **Archive Creation:** Uses archiver to create compressed .zip transfer packages, including all selected files and a metadata manifest describing the package contents and structure.
- **Metadata Management:** Generates and validates a transferMetaData.json manifest, ensuring every package is self-describing and versioned for compatibility.
- **Import & Extraction:** Uses adm-zip to extract transfer packages, validate their contents, and build a manifest mapping source files to their intended destinations in the app.
- **Validation & Cleanup:** Checks for missing or invalid files during import, and provides cleanup utilities for temporary extraction directories.

How It Works:

- When exporting, it collects files based on user options, creates a metadata manifest, and writes everything to a .zip archive.
- When importing, extracts the archive, validates the manifest and files, and prepares a file manifest for restoring data into the app.
- All file operations are robust, with error handling and clear user feedback for missing or invalid files.

Typical Usage:

- Used by the UI and backend logic to implement “Export Data”, “Import Data”, and backup/restore features. Ensures that users can safely transfer their work between installations or share with collaborators.

Notable Features:

- Supports granular selection of what to export/import (platforms, views, settings, filtered variables).
- Self-describing, versioned transfer packages for long-term compatibility.
- Robust error handling and validation for safe data migration.
- Clean separation of packaging, metadata, and file management logic.

integrations.js

integrations.js is the module that connects the JavaScript app to the Python processing layer. It is the JavaScript ↔ Python bridge.

NetSeaDF relies on Python for NetCDF parsing and scientific computations, while the UI and application flow are managed in JavaScript. This creates a cross-language boundary that must be handled reliably. Without a dedicated bridge module, Python calls would be inconsistent and difficult to debug.

integrations.js provides that bridge. It determines which portable Python runtime should be used based on operating system and architecture, starts a persistent Python process, and sends commands to it using a JSON request/response protocol over standard input/output. It also tracks pending requests, routes responses to the correct caller, and handles process errors and exits.

A key part of this design is that Python is kept alive as a long-running worker process rather than started fresh for every request. That improves performance and simplifies repeated dataset operations. The module also includes timeout handling for Python calls so the app can fail safely instead of hanging indefinitely.

By centralizing all JS-to-Python communication, integrations.js gives the rest of the app a clean `callPyFunc(...)` interface and hides process-management complexity. This keeps scientific processing integrated, stable, and maintainable.

Key Responsibilities:

- **Python Process Management:** Detects platform and environment (Windows, Intel Mac, Apple Silicon Mac, dev/production), selects the correct Python executable, and launches the Python worker process as needed.
- **Robust Communication:** Implements a JSON-over-stdin/stdout protocol for sending commands and receiving results from the Python process. Handles buffering, parsing, and error management for both stdout and stderr.
- **Request/Response Handling:** Maintains a queue of pending requests, matching each Python response to its corresponding JavaScript promise. Handles timeouts, process exits, and errors gracefully.
- **API Exposure:** Exports a single `callPyFunc` function, allowing other modules to invoke any supported Python command (e.g., `open`, `getVariable`, `bulkSSP`) with arguments and receive results as promises.

How It Works:

- On first use, launches the Python process with the correct executable and script path, setting up environment variables for portability.
- Sends JSON-encoded commands to the Python process's stdin, and listens for JSON responses on stdout.
- Each request is tracked with a promise; responses are parsed and routed to the correct handler.
- Handles process errors, exits, and timeouts, ensuring that all pending requests are resolved or rejected appropriately.

Typical Usage:

- Used by data, queue, and UI modules to perform all scientific data access and computation, offloading heavy work to Python. All calls to the Python backend (e.g., open dataset, extract variable, compute SSP) go through this integration layer.

Notable Features:

- Cross-platform, portable Python process management.
- Fully asynchronous, promise-based API for seamless integration with JavaScript code.
- Robust error and timeout handling for reliability.
- Clean separation of concerns: all Python/JS communication logic is isolated in this module.

pythonProcess.py

Acts as the persistent Python worker for the app, providing high-performance scientific data access and computation. It loads, processes, and serves NetCDF datasets using Python's scientific stack (xarray, numpy, gsw), and communicates with the Node.js/Electron app via JSON over stdin/stdout.

Key Responsibilities:

- **Dataset Management:** Opens, closes, and manages NetCDF datasets, keeping one dataset loaded at a time for efficient repeated access.
- **Data Extraction:** Provides functions to extract dimensions, attributes, variables, coordinates, and timestamps from scientific datasets, with robust handling of missing or non-standard fields.
- **Data Cleaning:** Converts numpy arrays and other scientific types into plain Python/JSON-serializable types, handling NaNs, infinities, and byte strings.
- **Advanced Computation:** Implements scientific calculations such as bulk sound speed profile (SSP) computation using the Gibbs SeaWater (gsw) library.
- **Flexible Variable Access:** Supports variable extraction by dimension, with options for slicing, reducing, and compact output.
- **Timestamp Formatting:** Converts and formats timestamps into various human-readable forms.
- **Persistent and One-Shot Modes:** Can run as a persistent process (for fast repeated queries) or as a one-shot command-line tool.

How It Works:

- Exposes a set of functions (open, close, getOverview, getVariable, bulkSSP, etc.) mapped to string commands.
- Listens for JSON commands on stdin, executes the requested function, and returns results as JSON on stdout.
- Handles errors gracefully, always returning a structured JSON response.
- Cleans all outputs to ensure compatibility with JavaScript and the Electron app.

Typical Usage:

- Spawned and managed by the Electron/Node.js backend. Used for all scientific data access, variable extraction, and heavy computation, offloading these tasks from the JavaScript side for speed and reliability.

Notable Features:

- Efficient, persistent process model for high-throughput data access.
- Robust handling of scientific data types and edge cases.
- Extensible command set for future scientific or data processing needs.
- Integrates seamlessly with the app's queue and integration logic for batch and interactive workflows.

DOM.js

DOM.js is the largest UI module. It builds sidebar entries, renders dataset detail panels, manages map markers/polylines/filters, constructs settings UI, and drives view/chart configuration panels. It is effectively the renderer-side “UI controller.”

Serves as the central hub for all DOM manipulation and UI rendering in the application. This module bridges the gap between the app’s data/state and the user interface, ensuring that all user interactions, data changes, and state transitions are accurately and efficiently reflected in the UI.

Key Responsibilities:

- **Sidebar Management:** Loads, updates, and styles the sidebar lists for glider data files and saved views. Handles dynamic font sizing and selection highlighting.
- **Map Marker Management:** Adds, removes, and updates Leaflet map markers, including advanced features like marker timelines, instance markers, polylines, and special flags (e.g., “under ice” events). Supports marker filtering by time and location, and manages marker opacity and state.
- **UI Feedback & Notifications:** Manages loading overlays, updates loading text, and provides functions to show/hide visual feedback during long-running operations. Integrates with the notification system to display banners and popups for user alerts, warnings, and errors.
- **Settings Menu:** Dynamically builds the settings menu from config data, supports live updates to settings, and manages section switching and UI state.
- **View & Chart Configuration:** Handles the creation, rendering, and management of view/chart configuration panels, including chart instance management, axis configuration, and output display. Supports dynamic resizing and animation of plot areas.
- **Utility Functions:** Provides reusable helpers for clearing, updating, and toggling DOM elements, managing visibility, and constructing or updating UI sections.

How It Works:

- Functions are called by the main UI logic and event handlers to update the interface in response to user actions (clicks, selections, navigation), data changes (file import, deletion), or state transitions (view switching, filtering).
- Integrates tightly with app state and other modules (fileHandle, config, queue, etc.) to ensure the UI always reflects the current state of the application.
- Uses modular, reusable patterns for all major UI components, making it easy to extend or modify the interface as the app evolves.

Typical Usage:

- Called when the user selects or deletes files, switches views, interacts with map markers, or changes settings.
- Used to update the sidebar, map, notification banners, loading overlays, and view/chart configuration panels in real time.

Notable Features:

- **Advanced Map Integration:** Supports complex marker and timeline logic, including multi-instance markers, polylines, and dynamic filtering.
- **Dynamic UI Construction:** Builds menus, lists, and configuration panels on the fly based on app state and config.
- **Responsive & Animated Layouts:** Ensures charts and views resize smoothly and remain usable across different window sizes and layouts.
- **Seamless State/UI Sync:** Maintains a tight connection between the app's data/state and the visible interface, minimizing UI bugs and inconsistencies.
- **Extensible & Maintainable:** Designed for easy extension, with clear separation of concerns and reusable utility functions for all major UI tasks.

userInteractions.js

Handles all user-driven actions and UI events related to data selection, map marker interaction, file deletion, and navigation within the app. This module acts as the bridge between user input (clicks, selections, button presses) and the app's state, data, and UI updates.

Key Responsibilities:

- Marker Timeline Toggle: Expands or collapses a map marker to show the full timeline of a platform's movement, adding/removing instance markers, polylines, and special flags (e.g., "under ice" events) based on timestamp analysis.
- Map Focus: Centers or pans the map to the currently selected file's main marker, supporting both single and multi-select navigation.
- File Deletion: Handles the deletion of selected data files, updating the UI, managing the remove queue, and ensuring all related UI elements and state are refreshed.
- Sidebar Navigation: Implements back/forward navigation through selected files, updating the dataset info cards and UI accordingly.

How It Works:

- Listens for user actions (button clicks, marker interactions) and updates the app state, UI, and data structures in response.
- Coordinates with other modules (DOM, fileHandle, queue, etc.) to perform complex UI updates, data processing, and asynchronous operations (like file deletion).
- Implements logic for advanced marker features, such as drawing timelines, adding numbered markers, and flagging unusual events (e.g., long gaps in timestamps indicating possible "under ice" periods).
- Ensures the UI remains consistent and responsive after user actions, including loading overlays, clearing info cards, and updating selection highlights.

Typical Usage:

- Called by event handlers in the UI layer when the user interacts with map markers, sidebar lists, or navigation buttons. Ensures that all user actions are reflected in both the app state and the visible interface, providing a seamless and interactive experience.

Notable Features:

- Sophisticated marker timeline expansion with visual cues for data anomalies.
- Multi-file selection and navigation support.
- Asynchronous file deletion with UI feedback.
- Modular design for easy extension of user interaction patterns.

charts.js

Provides all charting and plotting functionality for the app, using ECharts to render interactive scientific plots and timelines. This module transforms processed data into visualizations, manages chart configuration, and ensures responsive, user-friendly chart behavior.

Key Responsibilities:

- Plot Rendering: Builds and configures ECharts line/scatter plots for scientific data, supporting multiple axes, zooming, tooltips, and legends.
- Timeline Visualization: Renders an interactive timeline chart of dataset timestamps, allowing users to filter data by time range using zoom sliders.
- Color and Axis Management: Assigns consistent colors to files, manages axis scaling, padding, and formatting for clear, readable plots.
- Responsive Layout: Ensures charts resize automatically with window or container changes, maintaining usability across devices and layouts.
- Data Preparation: Zips and filters data arrays, handles missing/invalid values, and prepares series for plotting.
- Advanced Features:
 - Supports multiple X axes and dynamic axis labeling.
 - Custom tooltips showing file, variable, and timestamp details.
 - Data zoom controls for both axes.
 - Handles asynchronous plot readiness for smooth UI updates.

How It Works:

- Consumes processed data and chart/view configuration from the app state.
- Dynamically builds ECharts options for each chart instance, including axis, series, color, and interaction settings.
- Exposes `initTimeline` for timeline chart setup and `buildPlotInstance` for scientific plot rendering.
- Integrates with the DOM and app state to update plots in response to user actions, data changes, or window resizing.

Typical Usage:

- Called by the UI and view logic to render or update charts whenever the user changes views, selects files, or adjusts filters. Used for both the main timeline and all scientific data plots in the app.

Notable Features (charts.js):

- Highly configurable, supporting multi-axis, multi-file, and multi-variable plots.
- Interactive timeline for temporal data exploration.
- Robust handling of edge cases (missing data, single-point timelines, etc.).
- Modular and reusable for any charting needs within the app.

logging.js

Centralizes all logging, notification, and debug output for the app. This module provides a unified interface for recording events, errors, and warnings, and for displaying messages to users and developers via the console, popups, notifications, and log files.

Key Responsibilities:

- **Logging Events:** Records messages with labels, severity (info, warning, error), codes, and context. Supports structured log formatting for easy reading and debugging.
- **Console Output:** Outputs logs to the console, with severity-based formatting, if debug mode is enabled.
- **User Notifications:** Triggers in-app popups and notification banners for important events, warnings, or errors, ensuring users are informed of issues or actions.
- **File Logging:** Writes logs to persistent files in the logs directory, supporting later review and troubleshooting.
- **Log File Management:** Creates and manages log files, ensuring safe file naming and directory creation.
- **Flexible Output Control:** Each log can selectively enable/disable console, popup, notification, and file logging via parameters.

How It Works:

- The `log()` function is the main entry point, accepting a parameter object describing the message, label, seriousness, and output preferences.
- Builds a formatted log message with timestamp, caller info, severity, and code.
- Depending on parameters and config, routes the message to the console, popup, notification, and/or log file.
- Provides helpers for log file creation and management, and for building structured log messages.

Typical Usage:

- Called throughout the app to record significant events, errors, or warnings, and to inform users or developers of important actions or issues. Used by both infrastructure and UI modules to ensure all critical events are tracked and visible.

Notable Features (logging.js):

- Severity-based routing (info, warning, error) to different outputs.
- In-app notification and popup integration for user-facing alerts.
- Configurable debug mode for verbose logging during development.
- Safe, robust log file creation and writing.
- Extensible for future logging or notification needs.

objects.js

Defines shared data structures, enums, and variable mappings used throughout the app. This module provides a central place for common scientific variable aliases, time unit definitions, and the DataView class for managing view state and persistence.

Key Responsibilities:

- **DataView Class:** Represents a data view or graph, with properties for type, storage status, and a unique identifier (UUID). Supports serialization for saving/loading view state.
- **Variable Aliases:** Exports `varPossibilities`, a mapping of canonical scientific variable names (temperature, pressure, salinity, etc.) to all recognized aliases and NetCDF field names, enabling robust variable matching across datasets.
- **Time Unit Enum:** Exports `GraphType`, an object enumerating supported time units (nanoseconds, milliseconds, seconds, minutes, hours, days) for use in timestamp and interval calculations.

How It Works:

- The `DataView` class is instantiated with parameters describing the view; it generates or preserves a UUID for tracking and persistence.
- `varPossibilities` is used by data processing and UI modules to match user input or file fields to standard variable names.
- `GraphType` is used for consistent time unit handling in calculations, plotting, and data processing.

Typical Usage:

- Imported by modules that need to create or manage data views, match variable names, or perform time-based calculations. Ensures consistency and reduces duplication across the codebase.

Notable Features:

- Centralizes all variable aliasing and time unit logic.
- Provides a foundation for robust, user-friendly data handling and view management.
- Easily extensible for new variable types or time units as the app evolves.

popup.js

Implements a flexible, reusable popup framework for the app, supporting notifications, menus, and custom dialogs. This module provides the UI infrastructure for all modal interactions, alerts, and in-app messages, ensuring a consistent and user-friendly experience.

Key Responsibilities:

- **Popup Base Class:** Provides the core logic for creating, displaying, and managing modal popups, including headers, content, overlays, and close behavior.
- **NotificationPopup:** Specialized popup for info, warning, and error notifications, with icon support and auto-cleanup on close.
- **MenuPopup:** Popup with a dynamic menu bar, supporting the addition and removal of interactive menu items, each with custom click and delete handlers.
- **SimplePopup:** Minimalist popup for simple messages or dialogs.
- **Popup Templates:** Predefined templates for common popup scenarios (e.g., “No Files To Filter” warning).
- **Type and Notification Enums:** Lists of supported popup and notification types for consistent usage across the app.

How It Works:

- Popups are instantiated with configuration for content, callbacks, size, and style.
- The base class handles DOM creation, event binding, and show/hide logic.
- Specialized subclasses add icons, menus, or simplified layouts as needed.
- Popups can be opened, closed, updated, or deleted programmatically.
- Templates provide ready-to-use popups for frequent app events.

Typical Usage:

- Used by the logging, notification, and UI modules to display alerts, warnings, menus, and dialogs in response to user actions or system events. Ensures all modal UI is consistent and easy to manage.

Notable Features:

- Extensible class hierarchy for different popup needs.
- Integrated notification icons and menu support.
- Clean separation of content, behavior, and appearance.
- Ready-to-use templates for common scenarios.
- Robust event handling and DOM management for reliable user experience.

preload.js

Acts as the secure bridge between the Electron renderer process (web UI) and the main process, exposing safe APIs for data transfer operations and handling menu-triggered events. Ensures that only intended functionality is accessible to the web context, following Electron's security best practices.

Key Responsibilities:

- **IPC Bridge:** Uses Electron's `ipcRenderer` to expose a `transferApi` on the window object, allowing the UI to request data export via IPC.
- **Menu Event Handling:** Listens for custom menu events (e.g., `transfer`, `import`) sent from the main process, and routes them to handler functions in the renderer if available.
- **Deferred Event Support:** If a handler isn't yet registered, stores the event or payload on the window object for later processing.
- **Error Propagation:** Forwards import errors from the main process to the renderer, supporting both immediate and deferred error handling.

How It Works:

- Defines a `transferApi` with an `exportData` method, which invokes an IPC call to the main process.
- Listens for menu events and invokes corresponding handler functions if present, or stores the event for later if not.
- Ensures that only whitelisted APIs and events are exposed to the renderer, maintaining app security.

Typical Usage:

- Loaded as the Electron preload script for the renderer window. Used by the UI to trigger data export/import and to respond to menu actions initiated from the main process.

Notable Features:

- Follows Electron's security model for context isolation.
- Supports both immediate and deferred event handling for robust UI initialization.
- Cleanly separates main/renderer communication logic from UI code.

basicFunctions.js

Provides a collection of utility functions for variable name matching, time formatting, timestamp calculations, array navigation, and general-purpose helpers. Centralizes common logic used throughout the app for data processing, UI navigation, and time handling.

Key Responsibilities:

- Variable Name Matching: Functions to normalize, alias, and match variable names across different naming conventions and datasets.
- Time and Date Utilities: Formatting functions for 12hr/24hr time, timestamp difference calculations (supporting multiple units), and timestamp generation.
- Array and Navigation Helpers: Functions for clamping values, cycling through array indices, and calculating in-between coordinates.
- Notification and Logging Support: Helpers for determining notification seriousness and extracting caller info for logging.
- General Utilities: Includes functions for finding matching variables in arrays and mapping actions to UI elements.

How It Works:

- Exports each utility as a named function for use by other modules.
- Leverages the objects.js module for variable aliasing and time unit enums.
- Designed for reusability and consistency across the codebase.

Typical Usage:

- Imported by modules that need to match variable names, format or compare timestamps, navigate arrays, or perform common UI/data operations.

Notable Features:

- Robust variable aliasing and matching for scientific data.
- Flexible timestamp difference calculation with unit support.
- Clean, reusable helpers for both UI and data logic.
- Supports both immediate and deferred event handling for robust UI initialization. provides shared utility helpers: clamping, timestamp formatting, index wrapping, caller info extraction, and variable name matching against alias lists. It is a generic utility module used by many others.

Handling of application state

NetSeaDF manages its application state through a centralized, mutable `appState` object that acts as the single source of truth for the UI, data, and user interactions. This state object is passed throughout the codebase—especially to UI, DOM, and logic modules—ensuring that all parts of the application remain synchronized and responsive to user actions and data changes.

Key Principles:

Centralization: All major aspects of the app’s current status—such as selected files, loaded datasets, map markers, current view configuration, and UI mode—are stored in <code>appState</code>. This centralization allows for easy access and modification from any module that needs to react to or update the application’s status.	Mutability and Reactivity: <code>appState</code> is a mutable object, meaning its properties can be updated in place as the user interacts with the app (e.g., selecting files, switching views, filtering data). UI modules and event handlers read from and write to <code>appState</code> to ensure the interface always reflects the latest state.
Passing by Reference: Rather than using global variables, <code>appState</code> is passed as an argument to functions and modules that need to read or modify the state. This approach supports modularity and testability, while still allowing for real-time updates across the app.	Integration with UI and Data Logic: The state object is tightly integrated with DOM manipulation, data processing, and backend communication. For example, when a user selects a file, <code>appState.selectedFiles</code> is updated, which then triggers UI updates (highlighting, info cards, map markers) and may initiate data loading or processing.
Persistence and Synchronization: While much of <code>appState</code> is in-memory and session-based, key data (like loaded datasets, views, and user preferences) are persisted to disk (e.g., in <code>simpleData.json</code> or view files) and reloaded into state on app startup or after relevant operations. This ensures continuity across sessions and after batch operations like import/export.	Extensibility: The design of <code>appState</code> is flexible, allowing new features or UI components to add their own state properties as needed. This extensibility supports ongoing development and scaling of the application.
Typical Usage: UI event handlers update <code>appState</code> in response to user actions. DOM and rendering modules read from <code>appState</code> to update the interface. Data and backend modules use <code>appState</code> to determine what data to load, process, or display.	Summary: The handling of application state in NetSeaDF is designed for clarity, modularity, and real-time responsiveness. By centralizing all critical status and configuration in a single, mutable object, the app ensures that the UI, data, and backend logic remain in sync.

The following is a code snippet of the entire appState data structure.

```
const appState = {
  allFiles: fileHandle.listSavedDataFiles(pathDep.resolveToProperDataPath(__dirname, 'savedData'), '.nc'),
  isMultiSelect: false,
  selectedFiles: new Set(),
  selectedViewFile: null,
  currentFileIndexTarget: null,
  map: undefined,
  markers: {},
  filteredMarkers: [],
  mapTimelineMode: "slider", // or "input"
  mapTimeline12hrClock: false,
  mapTimelineRange: {
    slider: [],
    input: []
  },
  mapLocationFiltering: {
    locationFilterMap: null,
    locationFilterBounds: null,
    latMin: -90,
    latMax: 90,
    lonMin: -180,
    lonMax: 180
  },
  charts: {
    timeline: null
  },
  popups: {},
  notificationsSeriousnessArray: [],
  currentLog: null,
  currentViewport: 'map',
  filteredAxisVarBlacklist: [],
  viewColumnVisible: true,
  hasViewBeenCreated: false,
  currentView: {
    name: null,
    type: null,
    dataSelection: null,
    chartInstances: [],
    vars: null,
    viewVars: new Set(),
    isViewGenerated: false,
    includePlotData: false
  }
}
```

Python + JavaScript Integration

NetSeaDF achieves seamless integration between its JavaScript/Electron frontend and Python scientific backend through a robust, asynchronous, and platform-aware communication layer. This design leverages the strengths of both ecosystems: Python for scientific data processing (NetCDF, xarray, numpy, gsw) and JavaScript for UI, orchestration, and application logic.

Key Principles:

Persistent Python Worker:

A dedicated Python process (`pythonProcess.py`) is launched and managed by the Node.js backend. This process remains alive throughout the app session, enabling fast, repeated access to scientific data and computations without the overhead of starting a new process for each request.

JSON-over-stdin/stdout Protocol:

Communication between JavaScript and Python is handled via standard input/output streams. JavaScript sends JSON-encoded commands (with function names and arguments) to Python's `stdin`, and Python responds with JSON-encoded results or errors on `stdout`. This protocol is simple, language-agnostic, and robust.

Promise-based API:

The JavaScript integration layer (`integrations.js`) exposes a single `callPyFunc` function, which returns a promise for each Python call. This allows the rest of the app to use `async/await` patterns, making integration with UI and data logic straightforward and non-blocking.

Request/Response Matching:

Each outgoing request is tracked in a queue. As Python responses arrive, they are parsed and matched to their corresponding JavaScript promises, ensuring that results and errors are delivered to the correct caller.

Platform and Environment Awareness:

The integration layer detects the current OS, architecture, and development/production mode, selecting the correct Python executable and script path for each environment (Windows, Intel Mac, Apple Silicon Mac, dev/prod).

Error and Timeout Handling:

The integration layer includes robust error handling for process exits, timeouts, and communication errors. If the Python process crashes or becomes unresponsive, all pending requests are rejected, and the process can be restarted as needed.

Extensible Command Set:

The Python backend exposes a set of functions (open, close, getVariable, bulkSSP, etc.) that can be called from JavaScript. Adding new scientific capabilities is as simple as adding a new function to the Python command map.

Typical Usage:

All scientific data access, NetCDF parsing, and heavy computation are performed in Python.

The JavaScript app calls callPyFunc to request data or computations, and updates the UI or state with the results.

Batch operations (import/export, queue processing) and interactive features (map markers, charting) all rely on this integration for data extraction and analysis.

Summary:

The Python + JavaScript integration in NetSeaDF is designed for performance, reliability, and extensibility. By maintaining a persistent Python worker and using a clean, promise-based communication protocol, the app combines the best of both languages—delivering a responsive, feature-rich user experience with powerful scientific capabilities under the hood.

Anatomy of calling a PY function from a JS frontend

- High-Level Flow:
 1. User Action or App Logic triggers a need for scientific data or computation (e.g., loading a NetCDF file, extracting a variable, computing a sound speed profile).
 2. JavaScript Frontend calls a function (e.g., `callPyFunc('getVariable', ['TEMP'])`) from the `integrations.js` module.
 3. Integration Layer serializes the request as JSON and writes it to the Python process's stdin.
 4. Python Worker receives the command, executes the corresponding function, and writes the result (or error) as JSON to stdout.
 5. Integration Layer parses the response, resolves or rejects the associated JavaScript promise, and returns the result to the frontend/UI.

Detailed Step-by-Step Example

A. JavaScript: Making the Call

Suppose the frontend wants to get the values of the variable TEMP from the currently loaded NetCDF dataset.

```
const { callPyFunc } = require('./integrations.js');

async function fetchTemperature() {
  try {
    const result = await callPyFunc('getVariable', ['TEMP']);
    console.log('Temperature data:', result);
    // Use result to update UI, charts, etc.
  } catch (err) {
    console.error('Failed to fetch variable:', err);
    // Show error to user
  }
}
```

B. Integration Layer: Serializing and Sending the Request

callPyFunc prepares the request and ensures the Python process is running:

```
function callPyFunc(funcName, args = [], options = {}) {
  return new Promise((resolve, reject) => {
    // ...setup omitted for brevity...
    const req = JSON.stringify({ cmd: funcName, args: args });
    pyProc.stdin.write(req + "\n"); // Send to Python's stdin
    // ...response handling omitted...
  });
}
```

(Example request sent to Python)

```
{"cmd": "getVariable", "args": ["TEMP"]}
```

C. Python Worker: Receiving and Executing the Command

The Python process reads each line from stdin, parses the JSON, and dispatches to the correct function:

```
for line in sys.stdin:
    line = line.strip()
    if not line: continue
    try:
        req = json.loads(line)
        fn = functions.get(req.get("cmd"))
        if not fn: raise ValueError(f"Unknown cmd:
{req.get('cmd')}")
        data = fn(*req.get("args", []))
        response = json.dumps({"ok": True, "result": data}) + "\n"
        sys.stdout.write(response)
        sys.stdout.flush()
    except Exception as e:
        response = json.dumps({"ok": False, "error": str(e)}) +
"\n"
        sys.stdout.write(response)
        sys.stdout.flush()
```

The functions dictionary maps command names (like "getVariable") to actual Python functions.

D. Python Worker: Returning the Result

If successful, the Python function returns the result (e.g., a list of temperature values), which is wrapped in a JSON response:

```
{"ok": true, "result": [12.3, 13.1, 14.0, ...]}
```

If there's an error (e.g., variable not found):

```
{"ok": false, "error": "Variable 'TEMP' not found"}
```

E. JavaScript: Receiving and Handling the Response

The integration layer reads stdout, parses the JSON, and resolves or rejects the original promise:

```
pyProc.stdout.on('data', (chunk) => {  
  // ...buffering logic...  
  let payload = JSON.parse(line);  
  const next = pending.shift();  
  if (payload.ok) next.resolve(payload.result);  
  else next.reject(new Error(payload.error || 'Python error'));  
});
```

Key Features and Best Practices:

- **Promise-based:** All calls are asynchronous and return promises, making them easy to use with `async/await`.
- **Error Handling:** Both Python and JS sides wrap errors in a consistent JSON format, so errors are always catchable in JS.
- **Timeouts:** Each call can specify a timeout; if Python doesn't respond in time, the promise is rejected.
- **Extensibility:** To add a new Python function, simply add it to the functions dictionary in `pythonProcess.py` and call it from JS.

In summary:

Calling a Python function from the JS frontend in NetSeaDF is a robust, asynchronous, and extensible process, leveraging a persistent Python worker, a simple JSON protocol, and a promise-based API for seamless scientific computation and data access.

simpleData.json (Dataset Data Representation)

simpleData.json is a core data file in NetSeaDF, acting as a lightweight, persistent database for metadata about all loaded or imported NetCDF platform files. It is managed and accessed by the JavaScript backend (primarily via fileHandle.js and related modules) and is essential for efficient data management, UI rendering, and application state recovery.

The reason why this file exists is that NetCDF files can be rather complex and large in file size. This leads to latency issues when information about the file needs to be accessed. Due to the fact that in NetSeaDF, there is a near infinite number of instances where something about a .nc file will have to be known, constantly loading the file in and out of memory with recurring latency each time information is accessed is not feasible for the app. There needs to be a way to reference the information in an .nc file quickly without any of the performance overhead. This is the purpose of the simpleData.json.

This file acts as a lookup table for each .nc file that has been imported into the application. The way this works is as follows,

- When the user imports an .nc file, it will add the file to the app, then load the file into memory. When it does this, it retrieves the following information.
 - fileName
 - Summary (derived from attributes)
 - Dims
 - Vars
 - coords
 - Max pressure value (but only for the first profile in file)
 - Timestamps
 - All attributes
- When common information (like a files dimensions and variables) are needed, instead of reloading the .nc file into memory, the app instead gets the data from the files entry in the simpleData.json file.
- By using this approach, it allows for very easy lookup of a file's basic information.

(Actually, believe it or not, NetSeaDF files are only actually opened and read in two different scenarios. Importing a file, and generating a plot).

Key Roles and Structure for simpleData.json:

Purpose:

Serves as a fast-access index of all NetCDF datasets (“platforms”) currently available in the application. Stores essential metadata for each file, such as filename, coordinates, available variables, dimensions, attributes, and possibly summary statistics or user annotations.

Enables the app to quickly populate the sidebar, map markers, and data views without re-parsing large NetCDF files on every launch.

Format:

- JSON array or object, with each entry representing a single dataset/platform.
- Typical fields per entry:
 - fileName: The name of the NetCDF file.
 - coords: Array of coordinate objects (lat/lon, possibly time).
 - vars: List of variable names available in the file.
 - dims: Dimensions present in the file.
 - attrs: File-level attributes.

Lifecycle and Management:

- Updated whenever a new NetCDF file is imported, deleted, or modified.
- Used to check for desynced files (files present on disk but missing from the index, or vice versa).
- Accessed by the UI to display dataset info, populate map markers, and drive filtering/search.
- Supports batch operations, such as cleanup of orphaned files or bulk metadata updates.

Integration:

- Closely tied to fileHandle.js, which provides functions to read, write, and update simpleData.json.
- Used by DOM.js and other UI modules to render dataset lists, info cards, and map overlays.
- Ensures that the app can recover its state and present all available data to the user after restart or crash.

Role in the Application:

simpleData.json is the backbone of NetSeaDF's data management layer, providing a performant, persistent, and easily extensible way to track all datasets and their metadata. It decouples heavy NetCDF parsing from the UI, enabling fast startup, responsive filtering, and robust state management.

The following is a full code snippet of a files entry in simpleData.json

```
{
  "id": "1779640352505",
  "fileName": "GL_PR_PF_1902579.nc",
  "summary": "ARVOR Profiling Float",
  "dims": {
    "TIME": 3,
    "DEPTH": 566
  },
  "vars": [
    "TIME_QC",
    "POSITION_QC",
    "DC_REFERENCE",
    "DIRECTION",
    "PRES_QC",
    "PRES_ADJUSTED",
    "PRES_ADJUSTED_QC",
    "TEMP",
    "TEMP_QC",
    "TEMP_ADJUSTED",
    "TEMP_ADJUSTED_QC",
    "PSAL",
    "PSAL_QC",
    "PSAL_ADJUSTED",
    "PSAL_ADJUSTED_QC",
    "PRES_CORE"
  ],
}
```

```

"coords": [
  {
    "lat": 68.71810913085938,
    "lon": -12.184120178222656
  },
  {
    "lat": 68.76056671142578,
    "lon": -11.833769798278809
  },
  {
    "lat": 68.62239837646484,
    "lon": -11.773650169372559
  }
],
"lastPressureValue": 1827.1,
"timestamps": {
  "ms": [
    1764167370000000000,
    1765005510000000000,
    1765857270000000000
  ],
  "formatted": [
    "2025-11-26 14:29",
    "2025-12-06 07:18",
    "2025-12-16 03:54"
  ]
},
"attributes": {
  "platform_code": "1902579",
  "platform_name": "ARVOR Profiling Float",
  "wmo_platform_code": "1902579",
  "ices_platform_code": " ",
  "coriolis_platform_code": "1902579",
  "comment": " ",
  "transmitter_system": "IRIDIUM",
  "start_mission_date": "2022-06-15T13:30:00Z",
  "source": "drifting subsurface profiling float",
  "source_platform_category_code": "46",
  "institution": "Institute of Marine Research",
  "institution_edmo_code": "1351",

```

```

"institution_country": "Norway",
"institution_references": " ",
"naming_authority": "Copernicus Marine In Situ",
"title": "Global Ocean - In Situ Observation Copernicus",
"summary": " ",
"data_mode": "M",
"wmo_instrument_type": "844",
"area": "Global Ocean",
"geospatial_lat_min": "68.62240",
"geospatial_lat_max": "68.76057",
"geospatial_lon_min": "-12.18412",
"geospatial_lon_max": "-11.77365",
"geospatial_vertical_min": "0.00",
"geospatial_vertical_max": "1857.00",
"time_coverage_start": "2025-11-26T14:29:30Z",
"time_coverage_end": "2025-12-16T03:54:30Z",
"cdm_data_type": "profile",
"featureType": "trajectoryProfile",
"bottom_depth": " ",
"last_date_observation": "2025-12-16T03:54:30Z",
"last_latitude_observation": "68.62240",
"last_longitude_observation": "-11.77365",
"references": "http://marine.copernicus.eu http://www.marineinsitu.eu",
"license":
"https://marine.copernicus.eu/user-corner/service-commitments-and-licence",
"update_interval": "P1M",
"citation": "These data were collated within the Copernicus Marine Service (In Situ)
and EMODnet collaboration framework. Data is made freely available by the Copernicus Marine
Service and the programs that contribute to it.",
"doi": "https://doi.org/10.13155/59938 https://doi.org/10.13155/40846
https://doi.org/10.13155/53381 https://doi.org/10.13155/36230
https://doi.org/10.13155/43494",
"creator_name": "Kjell Arne MORK",
"id": "GL_PR_PF_1902579",
"netcdf_version": "netCDF-4 classic model",
"format_version": "2.0",
"Conventions": "CF-1.11 Copernicus-InSituTAC-FormatManual-2.0.0
Copernicus-InSituTAC-ParametersList-3.3.0 Copernicus-InSituTAC-AttributesList-1.0.0",
"date_modified": "2025-12-18T10:54:00Z",
"history": "2025-12-18T10:54:00Z : Creation",

```

```
"publisher_email": "cmems-service@ifremer.fr",  
"publisher_name": "Copernicus Marine Service",  
"publisher_url": "https://marine.copernicus.eu/ http://www.marineinsitu.eu/",  
"publisher_institution": "Ifremer"  
}  
,
```

Use of AI in NetSeaDF

AI is used extensively in NetSeaDF. This application was developed by a single developer, (with beginner level JavaScript experience), over the span of roughly 5 months. This entire application would not have been possible without the use of AI.

AI: Models Used

- Programming
 - Claude Opus 4.6
 - Claude Sonnet 4.6
 - GPT 5.4
 - GPT 4.1
- Learning and Planning
 - Gemini (Research Mode / Deep Thinking)

Very often in the beginning of the applications development, I would go into gemini and start asking questions about if the way I was about to approach a coding challenge was in fact a good approach. Sometimes Gemini claimed there were better ways to approach what I was doing, other times I was spot on. The importance is taking all of the information you have and making the best decision available to you. This was my approach.

Due to AI in 2026 struggling with the big picture when coding things, I circumvented AI misunderstanding by forcing the AI to think about things at the level of individual features. Once those features were added, then I would connect them, so I remained in control.

Although a substantial amount of the code (~70%) was generated by AI, Not a single line of code was ever integrated into the codebase without me knowing what it did and verifying its purpose.

Typical questions involved determining which file format should be used for user settings and configuration files. (originally I had decided on .ini files until Gemini detailed the advantages of .yaml files so I ultimately decided on YAML for user settings).

AI: Context

As of 2026, it is common practice when using AI agents for programming to provide the agent a file that contains the context of the entire project. As in, providing the agents a file that summarizes every detail possible for your project. This not only involves files in the project and what they do, but also coding standards the project is adhering to and design patterns that are guiding the project's design.

The importance of this cannot be overstated. Providing AI agents a context file allows them to better understand the project they are making changes to.

The following is NetSeaDF's full context file used throughout development to help guide the AI agents in a more regulated and informed direction during development.

The following file is included in this project for the purpose of providing context to everything about the project.

The following context will be provided,

- * What This project is and its purpose
- * The Features and functionality the app allows users to perform
- * The technology / tech stack this application relies on
- * additional tech notes
- * Simple overview of the code base (file structure)
- * Advanced overview of the code base (file structure of the source code (src) folder)
- * Technical source code overview (purpose of each source code file within src/common)

Project and its purpose

This project is a cross-platform electron-based desktop application called NetSeaDF. As you can tell due to the slight play-on-words, this project is designed around NetCDF files.

The purpose of this application is to provide an application for scientists to visualize NetCDF data on a map as well as plot their data on custom graphs.

The following expectation can be constructed, "Visualizing locations of NetCDF files on a map, filtering our available files and plotting graphs of the data that include Depth, Temp, Psal and SSP". If this application can allow scientists to do this, then this application has served its purpose.

Through the use of this application, there is a hope that users of this app will be able to achieve better data visualization and data discovery by interfacing with what this application offers.

Features and Functionality

The following is a list of the (current) features that have been implemented within the application,

- * Importing NetCDF Files.
- * Viewing a "Virtual File System" of all currently imported files.
- * Clicking on an imported file and viewing its Vars, Dims, Coords as well as Attrs.
- * Viewing markers on a map once a file is imported.
- * Clicking on a map marker (which resembles a dataset) to view that datasets information (especially coords).
- * Viewing the timeline of a current map marker to view all timestamps and their coord locations.
- * Being informed of the possibility of UnderIce loss of transmission through the feature of adding a map marker, informing the user of such a possibility if a marker includes a suspicious amount of time between timestamps.
- * Filtering map markers by timestamp via the use of a timeline range.
- * The ability to instead filter by a different type of timeline range using manual datetime-local inputs.
- * The ability to convert time displays to 12 or 24 hour variants.
- * Focusing on a map markers location by pressing the Focus button
- * Multi-selecting more than one file and cycling through the current set of selected files, in-turn, allows you to view the data and focus on a desired file that happens to be selected.

Technology / Tech Stack

The following is a list of the languages used in the project and what they are used for.

- * Languages
 - * Python (most used on the backend for working with the NetCDF Data)
 - * JavaScript (Makes up the majority of the code including the backend with file handling and also is fully responsible for the frontend.)
 - * C/C++ (only present due to the Python virtual environments)
- * Libraries
 - * Python
 - * GSW (handles the sound speed and other scientific equations)

- * xarray (main library for handling NetCDF files)
- * numpy (dependency for xarray)
- * sys, json, os, math, datetime (basic stuff / generic functionality)
- * JavaScript
 - * apacheEcharts (responsible for all of the apps charts and graphs)
 - * Electron (responsible for providing the environment within which the app exists)
 - * Leaflet.js (the main mapping library handling all maps and markers)
 - * path, fs (node modules for basic file handling)

Additional tech notes

* Electron is being used with node integration turned on. Despite the security concerns, this decision was made due to the fact that this application is meant for offline local use and does not need to concern itself with malicious network requests that would demand node be turned off. Plus it's much easier to make local resource requests directly so node integration is turned on.

- * There are 3 Python virtual environments within the file system, they are as follows,
 - * PythonPortable (Windows environment)
 - * PythonPortableMac_arm64 (Mac (apple) environment)
 - * PythonPortableMac_x86 (Mac (intel) environment)

The reason these 3 environments exist and are not ignored by the ignore file is so that the github actions script can bundle each executable with a compatible version of python and each library.

* The entire project has one main python file (pythonProcess.py). This file is holding all of the functions to perform actions on NetCDF files for the whole application. If you see a python file within the project's file system, it has another use that direct integration with the JS backend and frontend.

* Within the projects file system, there is a logs, config and savedData folder. These loose folders at the root level are the main folders for their use case, but only in development. In production, these 3 folders are still used, but the production builds reference the dist folder instead.

Therefore,

in dev, getting data would involve .savedData/

in prod, getting data would involve .dist/savedData/

* This application's features are made possible due to Python and JS working together. JS provides the application while python provides the data retrieval abilities for NetCDF files, facilitated through xarray.

The way this application parses NetCDF files is by allowing JS files to call the `callPyFunc()` function from integrations, allowing the ability to pass the python function name as a string argument to `integrations.js` which then interfaces with the python process, inturn, making python call the function, returning its data to `integrations`, inturn, allowing the original JS file to receive the python return through the `integrations` return.

Simple overview of the codebase

The following is a simplified list of the projects file structure and all its main folders.

- * (root)
 - * .github (holds the build.yml)
 - * associated with github commits
 - * build (holds the afterPack.js)
 - * afterPack hook for electron-builder
 - * Sets executable permissions on Python binaries for macOS builds
 - * config (holds config files)
 - * dist (holds the directories for distribution)
 - * This gets bundled with the app and is a folder exposed in the file system due to it holding the config and logs.
In other words, in a production build, all storage, config and logs go here
 - * This folder holds the config, logs and savedData folders.
 - * logs (holds the development build version of dist/logs)
 - * node_modules (holds JS dependencies)
 - * output (output folder from manual electron builds)
 - * PythonPortable (Windows environment)
 - * PythonPortableMac_arm64 (Mac (apple) environment)

- * PythonPortableMac_x86 (Mac (intel) environment)
- * queues (holds the code files for the queue functionality, both import and removal)
- * savedData (holds the development build of dist/savedData)
 - * this folder acts as the storage of the app, all imported files, including json data of file information is stored in savedData (both dev and prod builds)
- * src (main directory of all source code)
- * tooling (directory holding various code files which act as tools for debugging / testing)

Advanced overview of the codebase

The following gives context to each subfolder of the src folder

- * src
 - * common (all primary source code files that run the application)
 - * debug (files for debugging / testing)
 - * lib (external third-party libraries)
 - * media (holds all media like pictures and gifs)
 - * style (holds every main css file for the frontend of the app)
 - * unitTests (holds all previous unit tests of the app)
 - * map.html (The entire application, main.js calls this upon "npm start")

Technical source code overview

The following gives context to each primary source code file that runs the application. These files exist in src/common.

It is very, very important to understand the context of the applications code, and the overwhelming majority of the code that manages the frontend and backend is within the src/common folder.

- * basicFunctions.js (provides basic math and array functionality that any application might need)
- * charts.js (holds all code for charts, you can think of this as the "echarts file")
- * DOM.js (holds the majority of the code for manipulating the DOM)
- * fileHandle.js (holds functions for fundamental file handling operations)
- * integrations.js (the file responsible to allowing JavaScript to communicate with python via childProcess)

- * `logging.js` (provides functions for logging data to log files)
- * `objects.js` (provides fundamental objects the app relies upon and bases its data around)
- * `pathDep.js` (provides fundamental functionality for path traversal in the file system)
- * `preload.js` (a file to confirm that preloading works upon starting the app)
- * `pythonProcess.py` (the python file responsible to retrieving NetCDF data and sending it back to `integrations.js`)
- * `queue.js` (provides functions for import and remove queues)
- * `templates.js` (provides objects for the DOM, (this might soon be merged into `objects.js`))
- * `unitTest.js` (provides a small framework for performing unitTests)
- * `userInteractions.js` (provides functions for when users interact with DOM elements)

Special Thanks

Very Special Thanks to the Opera Lab for their support throughout the Program.

This project would not have become what it has if I never worked so hard and I never would have worked as hard as I did if I wasn't doing so, surrounded by such great people.

*Thank you,
Nick Fran – Software Engineering Intern*

